information

MDPI

Article

Exploring the Use and Misuse of Large Language Models

Hezekiah Paul D. Valdez *, Faranak Abri, Jade Webb and Thomas H. Austin

Computer Science Department, San José State University, San José, CA 95192, USA; faranak.abri@sjsu.edu (F.A.); jade.webb@sjsu.edu (J.W.); thomas.austin@sjsu.edu (T.H.A.)
* Correspondence: hezekiahpaul.valdez@sjsu.edu

Abstract

Language modeling has evolved from simple rule-based systems into complex assistants capable of tackling a multitude of tasks. State-of-the-art large language models (LLMs) are capable of scoring highly on proficiency benchmarks, and as a result have been deployed across industries to increase productivity and convenience. However, the prolific nature of such tools has provided threat actors with the ability to leverage them for attack development. Our paper describes the current state of LLMs, their availability, and their role in benevolent and malicious applications. In addition, we propose how an LLM can be combined with text-to-speech (TTS) voice cloning to create a framework capable of carrying out social engineering attacks. Our case study analyzes the realism of two different open-source TTS models, Tortoise TTS and Coqui XTTS-v2, by calculating similarity scores between generated and real audio samples from four participants. Our results demonstrate that Tortoise is able to generate realistic voice clone audios for native English speaking males, which indicates that easily accessible resources can be leveraged to create deceptive social engineering attacks. As such tools become more advanced, defenses such as awareness, detection, and red teaming may not be able to keep up with dangerously equipped adversaries.

Keywords: AI misuse; cybersecurity; digital assistant; generative AI; large language models; voice cloning

check for updates

Received: 28 July 2025
Revised: 21 August 2025
Accepted: 27 August 2025
Published: 1 September 2025

Citation: Valdez, H.P.D.; Abri, F.; Webb, J.; Austin, T.H. Exploring the Use and Misuse of Large Language Models. *Information* **2025**, *16*, 758. https://doi.org/10.3390/info16090758

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Large language models (LLMs) are artificial intelligence (AI) architectures that have evolved beyond the idea of simply communicating with technology [1]. When connected to other systems, LLMs help millions of people globally answer questions, summarize information, communicate with others, improve processes, and automate systems. These frameworks, often referred to as chatbots, generative AI (GenAI), or digital assistants, are becoming more widespread in their contributions to improving workflows and providing general assistance in a variety of fields. Assistants such as GitHub Copilot [2] (beta release, July 2021) or Phind [3] provide immense value in the domain of programming, as they support vital improvements in individual programmer productivity. Generative frameworks also demonstrate great potential in the field of medicine, due to their ability to assist doctors and patients in the distillation of comprehensive health profiles [4]. Microsoft itself has asserted their mission [5] to expand their Copilot platform across a vast ecosystem of domains.

As digital assistants become increasingly ubiquitous, malicious actors are beginning to leverage their generative capabilities for nefarious purposes [6–12]. Despite the moderation

safeguards embedded within popular models, such as ChatGPT (as described in OpenAI's GPT-4 Technical Report [13]), it is still possible for threat actors to utilize these services to commit acts of fraud [14], privacy breaches [15], and other criminal activity [16].

The contributions of this paper are as follows:

- Provide a background on what constitutes an LLM, how they evolved, and how their efficacy is measured.
- Provide examples of existing LLM-based GenAI models and their applications, both benevolent and malicious.
- Demonstrate how threat actors can leverage easily accessible GenAI tools to develop complex social engineering attacks.

2. What Is a Large Language Model

An LLM is a classification of GenAI model that is capable of solving complex natural language processing (NLP) tasks. When these models are specialized and incorporated into larger systems, they can be used as a digital assistant, also known as a copilot [5,17]. Copilots have been described by Microsoft as "[a tool] that brings the right skills to you when you need them" [5]. These assistants operate by intelligently interpreting user inputs and context to determine the most viable output or action. This equips them to manage a host of workflows for task simplification.

Digital assistants were initially created as an experiment to evaluate how a human would communicate with a machine [1]. In 1966, MIT's ELIZA [18] achieved this feat by using a rule-based system. ELIZA users would type sentences on a typewriter and ELIZA would scan for specific keywords. These ranked keywords would then be used to transform the input sentence into a response. ELIZA was limited by grammar rules, such as commas, and "only transformed specific phrases or sentences" [18]. ELIZA was highly limited by the basic technology of its time and its lack of establishing proper context. ELIZA knew nothing about the outside world and could not comment on it. Instead, context was provided through a "script" that gave ELIZA a set of responses to keywords and a set of sentence structures that guided it to respond in a specified manner. This "script" was the heart of ELIZA and provided it with artificial rule-based and response-driven context.

This rule-based approach was a simple yet limited way for a computer to process language. Eventually, more complex statistically based mathematical language models, such as n-gram models and bag of words, were created to analyze responses based on a compilation of text known as a corpus [19]. Neural networks eventually replaced these systems, as they provided even better probabilistic-based results. Models such as Recurrent Neural Networks (RNNs) and Long Short-Term Memory Networks (LSTMs) could solve textual problems that required greater contextual understanding [20]. However, these models still had speed and size limitations that limited their success.

In 2017, the transformer [21] pioneered a huge leap in the field of AI. Figure 1 illustrates the proposed attention-based transformer. This architecture differentiates itself from its predecessors because it does not leverage backpropagation from RNNs or LSTM units. Instead, it uses an encoder–decoder mechanism dependent on self-attention. Self-attention uses the relative positions of elements in an input sequence to generate an output while focusing on important parts. The encoder uses a six-layer network, with each layer containing a feed-forward network connected to a self-attention layer. Then, the result of this encoder is passed to a decoder that possesses the same architecture as the encoder, with another attention layer added to process the current output and the encoder output to generate the result. Since this transformer model relies on self-attention instead of backpropagation, the distance from any dependency is only O(1). It also does not require a sequential stream of inputs as in RNNs. These robust abilities allow larger corpora to be

processed, thereby making the transformer the architecture of choice to power many LLMs and subsequent copilots.

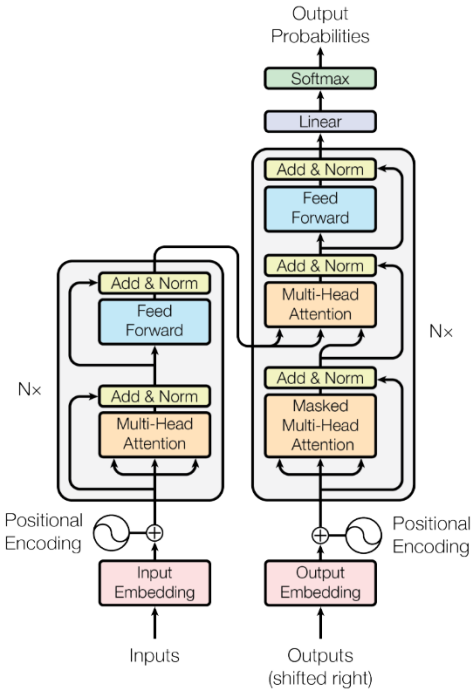

Figure 1. Fundamental Transformer Architecture, Adapted from [21], Licensed under Apache 2.0.

One negative aspect of LLMs is that they require billions of parameters, as well as tremendous amounts of time and computing power [13,22,23]. However, improvements in quantization have helped reduce the overall size and training time of LLMs. Quantization is the process of converting a high-precision representation to a smaller discrete value (e.g., a conversion from a floating-point representation to an 8-bit representation). This conversion leads to a loss in accuracy but an increase in performance (as is evident in smaller representations). Quantization methods such as those described by Jacob et al. [24] and Xiao et al. [25] demonstrate that improving quantization on weights and biases can permit large models to run on smaller form factors. Jacob et al. [24] determined that arithmetic quantization of both weights and biases can improve the running speed of a face detection model so that it is able to run in real-time with only a 10% reduction in accuracy. Where Jacob et al. [24] focused on weights and biases, Xiao et al. [25] focused on reducing the size of the activation functions by "smoothing" activations with a per-channel smoothing factor. When evaluating these methods on Llama models, using 8-bit representations for the weights and activation functions produced the same or similar perplexity scores, indicating that little conversational meaning is being lost.

2.1. Benchmarks

To understand existing LLMs, it is important to understand how the efficacy of each model is measured. For humans, a standardized test is often used to measure an individual's skill in a particular field. In a comparable manner, benchmark datasets are used to measure the efficacy of a model regarding a particular skill or domain. These datasets can measure an LLM's proficiency in inference, context, and other areas of conversation and reasoning. Some example datasets are the HellaSwag dataset by Zellers et al. [26], which was created with the explicit purpose of measuring the ability of a chatbot to perform natural language inference; the LAnguage Modeling Broadened to Account for Discourse Aspects (LAMBADA) dataset by Paperno et al. [27], which was created to evaluate how

well a language model can maintain context over the course of extended dialogue; and the HumanEval dataset by Chen et al. [28], which was created to evaluate the programming effectiveness of LLMs.

2.2. Prompt Engineering

Traditionally, many AI models rely on data that was pruned manually or through an automated process [29]. However, modern LLMs such as GPT-4 [13] have been pre-trained on a highly extensive corpus of general data and can immediately apply that knowledge to generate outputs without further tuning. Furthermore, high scores on language comprehension and intent benchmarks demonstrate that LLMs can understand the context and content of their training data, and can provide them to the user when prompted. This "prompting" is a feature that can be leveraged to further constrain responses and guide LLMs to return answers that more closely align with user-defined outcomes.

A prompt is a block of text that is provided as instructional input to LLMs. LLMs interpret then extend the prompt with output text that it considers as the best completion of that prompt. This process can be repeated as many times as needed to generate the desired response, and often involves feeding the model past outputs in addition to the initial prompt. The task of developing and refining these prompts is not trivial, and is known as "prompting" or "prompt engineering" [30].

In a 2021 paper by Reynolds and McDonell [31], the authors described six techniques they used to design prompts that led to optimal responses for their LLM-based project. The first technique is to describe direct tasks by giving specific orders on what the LLM should be performing. Some examples named by the authors are "translate the following," "rephrase this sentence," and "reiterate this point." The second technique is the specification of example prompt/output pairs to provide context and help the LLM understand exactly what it should produce. The third technique is task specification by memetic proxy, or using cultural analogies to specify a complicated task. An example used by the authors is "how would an elementary school teacher explain this?" The fourth technique is constraining behavior to limit the possible inputs or outputs from a given prompt. The authors provided the following examples: "translate everything to the right of the colon" and "describe this in a single paragraph." The fifth technique is serializing reasoning by prompting an LLM to break down a task or problem into steps as it reaches a solution. This process is known as chain-of-thought reasoning, and often involves asking the LLM to "show work." The final technique, metaprompt programming, adds different intentions to existing questions. An example given by the authors involves the task of solving multiple choice questions. Instead of asking the LLM to solve the questions, the prompt could feed the questions and answers to the LLM and instruct it to analyze each answer.

3. Existing Models

Now that we have provided a background on LLMs, this section will describe several currently released models. These language models fall under one of two categories: open models and closed models. Open models are more widely available and are free to use and alter. These models often come with documentation that describes in detail their training methods, training data, and internal architecture. Furthermore, they often have their source code available. In contrast, closed models hide their internal architecture, and instead, present benchmark results to indicate efficacy. Selections of popular models in each category are presented below.

3.1. Open Models

Open-source models are released with the intention of long-term societal benefit [23,32]. These models aim for transparency in their presentation of problems and limitations encountered during creation, in hopes that other developers can avoid these issues. Although the base forms of these models are often outperformed by their closed-source counterparts [23], these releases help provide a baseline that can be followed and subsequently improved upon. Many of the models presented here can be found on HuggingFace.com [33] and tools like LLM studio [34].

Microsoft's DialoGPT [35] is an open-source extension of OpenAI's GPT-2, developed in 2020 by Zhang et al. The design intent behind DialoGPT was to create a system that would be able to generate more neural responses, or responses that are natural-looking but completely distinct from any training data. To achieve this, the authors leveraged GPT-2's stacked transformer architecture, which incorporates multiple transformers [21]. To model realistic conversation, they collected a dataset of Reddit comment chains from 2005 to 2017. To evaluate the efficacy of their model, the authors utilized the DSTC-7 Dialogue Generation Challenge as a test set with automatic translation metrics using BLEU [36], METEOR [37], and NIST [38]. When compared to the performance metrics of the winner of the DSTC-7 Dialogue Generation Challenge, DialoGPT performed better on all marks. Furthermore, when humans were asked to rank human Reddit responses and DialoGPT responses, many participants preferred generated responses to actual Reddit comments. DialoGPT is released as open source but still has a propensity to output offensive responses, potentially due to the nature of its training data.

Google's Flamingo is a Visual Language Model (VLM) introduced in 2022 by Alayrac et al. that is capable of answering a plethora of image- and video-based tasks based on only a few input/output examples [39]. As shown in Figure 2, Flamingo uses two important components in its architecture: the vision encoder and the perceiver resampler. The vision encoder is a Normalizer-Free ResNet that has been pre-trained on image–text pairs and then frozen. The Resnet takes input images and converts them into features to be passed to the perceiver resampler. The perceiver resampler processes these features and outputs a fixed number of visual outputs, which are passed to a transformer for analysis. To train Flamingo, the authors used the ALIGN image dataset coupled with their own dataset of long texts paired with images and videos. To evaluate Flamingo, the authors used a total of 16 benchmarks, on which Flamingo outperformed previous few-shot methods by a large margin. Additionally, after fine-tuning, Flamingo was able to achieve state-of-the-art benchmark scores on an additional five datasets: VQAv2, VATEX, VizWiz, MSRVTT-QA, and Hateful Memes.

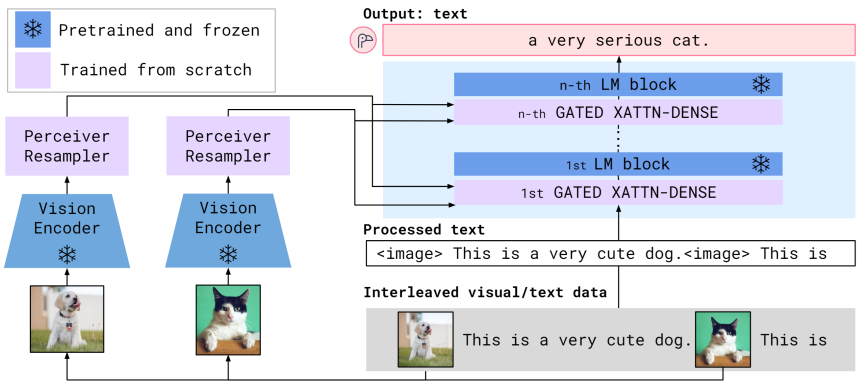

Figure 2. Google's Flamingo VLM Architecture, Adapted from [39], Licensed under Apache 2.0.

Meta's Llama 2 [23] is a family of open-source LLMs introduced by Touvron et al. in 2023. Llama 2 was pre-trained on two trillion tokens of data from publicly available data outside of Meta's own offerings. Llama 2 was created using supervised fine-tuning involving a dataset of annotated samples that include both the prompt and the response. In addition, the authors utilized Reinforcement Learning from Human Feedback (RLHF) to tune the model further. In this approach, human annotators prompted the model and selected the outputs that they preferred. Then, a number based on the prompt and response was generated to represent the quality of the interaction. This data, combined with open-source preference sets, was used to form the final dataset. Regarding efficacy, Llama 2 performed worse on common benchmarks when compared to stronger closed-source models, as shown in Figure 3.

Benchmark (shots)	GPT-3.5	GPT-4	PaLM	PaLM-2-L	Llama 2
MMLU (5-shot)	70.0	**86.4**	69.3	78.3	68.9
TriviaQA (1-shot)	–	–	81.4	**86.1**	85.0
Natural Questions (1-shot)	–	–	29.3	**37.5**	33.0
GSM8K (8-shot)	57.1	**92.0**	56.5	80.7	56.8
HumanEval (0-shot)	48.1	**67.0**	26.2	–	29.9
BIG-Bench Hard (3-shot)	–	–	52.3	**65.7**	51.2

Figure 3. Comparison of Benchmark Performances of Meta's Open-Source Llama 2 and Closed Models in the GPT and PaLM Series, Where Bold Values Indicate the Best Performance for Each Benchmark, Adapted from [23], Licensed under Apache 2.0.

Code Llama, presented by Alayrac et al. in 2023 [40], was developed to extend Llama 2 using the fine-tuning process shown in Figure 4. When evaluated against the HumanEval [28] benchmark for programming proficiency, Code Llama achieved a score of 67.8% compared to GPT-4's 67% [13]. However, these results come with the caveat that Code Llama used several different input sizes and a variety of training methods, such as fine-tuning exclusively on Python code and instruction following (as described in the Code Llama report). Code Llama combines infilling (filling in missing text) while considering surrounding context, long-input contexts, and instruction fine-tuning (using Llama 2 to generate code problems for Code Llama to solve) in different configurations to form three variants. Code Llama (base), Code Llama—Python (Python specialization), and Code Llama—Instruct (fine-tuned version with instruction-following).

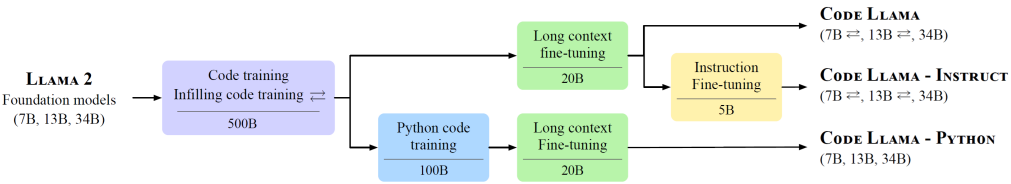

Figure 4. Fine-Tuning Process for the Code Llama Family of Models, Adapted from [40], under a Permissive License.

Since the Llama family is open source and available on HuggingFace.com [33], there are many derivative models that have improved upon base Llama forms through specialized fine-tuning. Phind is a programming-specific copilot that expands upon Code Llama [40]. Phind is an active competitor against ChatGPT, and has beaten GPT-4's HumanEval score for programming proficiency; GPT-4 achieved a score of 67% [13], while Phind achieved a score of 74.7% [3]. Phind also achieves an extensive computational speedup over GPT-4, leveraging both Nvidia's H100 cluster GPU and Nvidia's TensorRT-LLM library. By using these tools in conjunction, Phind can support one hundred tokens per second single-stream, 12k input tokens, and 4k tokens for web results. A negative aspect of Phind is that it occasionally requires more generations to achieve a solid result

when compared to GPT-4. According to Phind, this phenomenon only occurs "on certain challenging questions where it is capable of getting the right answer" [3].

Pathways Language and Image Model, or PaLI, is a model developed in 2023 by Chen et al. for image and text handling in a variety of languages [41]. The PaLI architecture consists of three separate models: one visual component based on ViT-G [42] (a zero-shot image-text model) and two transformer models trained on a variety of language understanding tasks. To train PaLI, the authors created their own dataset, called the WebLI dataset, which consists of Internet-sourced text and image data spanning over 109 languages. The authors evaluated PaLI's performance using a variety of benchmarks, including VQAv2, OK-VQA, TextVQA, VizWiz-QA, and ANLS*. PaLI achieved state-of-the-art results, as it outperformed some previous models, including Flamingo, "by +2.2 accuracy points on the open-vocabulary generation setting" [41].

Google's Gemma was developed in 2024 by Mesnard et al. [32] to expand upon their previous Gemini advances with the goal of achieving competitive performance in lightweight models. The Gemma family of models is composed of smaller versions of its older sibling, Gemini [22]. Gemma utilizes transformer architectures with several improvements, namely multi-query attention [43] (attention layers share a set of keys and values), RoPE Embeddings (rotary positional embeddings are used in each layer), a GeGLU activation function [44], and RMSNorm [45] (standardizes the transformer sublayer). Gemma was pre-trained on three trillion and six trillion tokens for the two billion and seven billion parameter model versions, respectively. This pre-training data was gathered from English sources that were filtered to remove unsafe words, utterances, and personal data. Unlike Gemini, Gemma's models do not expand upon previous multi-sphere projects like PaLI [41], as Gemma is non-multimodal and English-based. The authors utilized supervised fine-tuning techniques such as chain-of-thought prompting and RLHF to fine-tune the Gemma models. A portion of Gemma's performance metrics is shown in Figure 5. On average, Gemma 7B performs better on these assessed performance metrics, solving 56.9% of presented problems, while the next best model (Mistral 7B) solved an average of 54.5%.

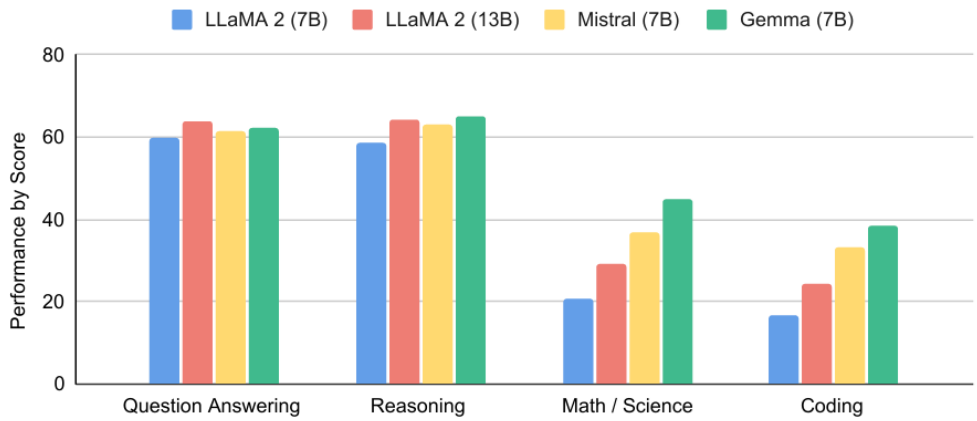

Figure 5. Comparison of Metric Performances Between Google's Gemma 7B and Other Open-Source LLMs, Adapted from [32], Licensed under Apache 2.0.

3.2. Closed Models

Aside from clear monetization intentions, closed models differ from their open-source counterparts because they are considered as black boxes. Much of their internal architecture remains obfuscated in comparison to the models mentioned in Section 3.1. Despite the lack of transparency, these models often outperform their open counterparts [23,40] on benchmark tests. The following is a selection of commonly used closed models.

OpenAI and their GPT series of models have become a ubiquitous standard for high-performing LLMs. Since their debut of ChatGPT [46], OpenAI has continuously improved their models to provide more in-depth responses and absorb more context. Their most recent model, GPT-4, is the general large-scale model that powers the current iteration of ChatGPT [13]. GPT-4 is a transformer-based model that was trained on a vast amount of publicly and privately sourced data. GPT-4 is multimodal, as it accepts both text and image inputs, yet is only capable of generating text outputs. For competitive reasons, OpenAI has not outlined GPT-4's architecture, specific training data, or implementation techniques. However, OpenAI conveys the power of GPT-4 by comparing its benchmark performance against that of its predecessor, GPT-3.5. This comparison is summarized in Figure 6, which illustrates GPT-4's superior performance in every tested category using few-shot learning. Two of the datasets discussed in Section 2.1, HellaSwag [26] and HumanEval [28], were used, with GPT-4 scoring 95.3% and 67% and GPT-3.5 scoring 85.5% and 48.1%, respectively. A study published in 2024 by Jones and Bergen asserts that when presented with conversational chats from GPT-3.5, GPT-4, ELIZA, and a human, participants identified GPT-4 as human half the time [47]. This demonstrates how advanced AI copilots have become and establishes GPT-4 as a strong leader.

	GPT-4 Evaluated few-shot	GPT-3.5 Evaluated few-shot	LM SOTA Best external LM evaluated few-shot	SOTA Best external model (incl. benchmark-specific tuning)
MMLU [49] Multiple-choice questions in 57 subjects (professional & academic)	**86.4%** 5-shot	70.0% 5-shot	70.7% 5-shot U-PaLM [50]	75.2% 5-shot Flan-PaLM [51]
HellaSwag [52] Commonsense reasoning around everyday events	**95.3%** 10-shot	85.5% 10-shot	84.2% LLaMA (validation set) [28]	85.6 ALUM [53]
AI2 Reasoning Challenge (ARC) [54] Grade-school multiple choice science questions. Challenge-set.	**96.3%** 25-shot	85.2% 25-shot	85.2% 8-shot PaLM [55]	86.5% ST-MOE [18]
WinoGrande [56] Commonsense reasoning around pronoun resolution	**87.5%** 5-shot	81.6% 5-shot	85.1% 5-shot PaLM [3]	85.1% 5-shot PaLM [3]
HumanEval [43] Python coding tasks	**67.0%** 0-shot	48.1% 0-shot	26.2% 0-shot PaLM [3]	65.8% CodeT + GPT-3.5 [57]
DROP [58] (F1 score) Reading comprehension & arithmetic.	80.9 3-shot	64.1 3-shot	70.8 1-shot PaLM [3]	**88.4** QDGAT [59]
GSM-8K [60] Grade-school mathematics questions	**92.0%*** 5-shot chain-of-thought	57.1% 5-shot	58.8% 8-shot Minerva [61]	87.3% Chinchilla + SFT+ORM-RL, ORM reranking [62]

Figure 6. Comparison of Benchmark Performances of OpenAI's GPT-4 and Other State of the Art Models, Including GPT-3.5. Bold Values Indicate the Best Performance for Each Benchmark, and an Asterisk (*) Denotes Chain-of-Thought Prompting as Reported in the GPT-4 Technical Report [13]. Adapted from [13], Licensed under Apache 2.0.

OpenAI has not limited itself to text generation. In early 2024, OpenAI published a technical report detailing their new video-based model, Sora [48]. Sora uses a variety of techniques that are synonymous with OpenAI's previous work. Firstly, Sora is a transformer-based model. It has an encoder that accepts video input and transforms it into a lower-dimensional representation, and a trained decoder that converts this representation back into pixels that can be used to build a new video. These lower-dimensional representations, known as patches, are analogous to the text tokens of LLMs, as shown in Figure 7. Sora is also capable of multimodal prompts. Using DALL·E 3 [49], OpenAI created a labeled training set of text inputs to train Sora to generate video based on text. To improve Sora's

language understanding, OpenAI leveraged Generative Pre-trained Transformers (GPTs) to extend short prompts and provide a larger description corpus. In addition to video and text inputs, Sora is able to generate videos based on static image inputs.

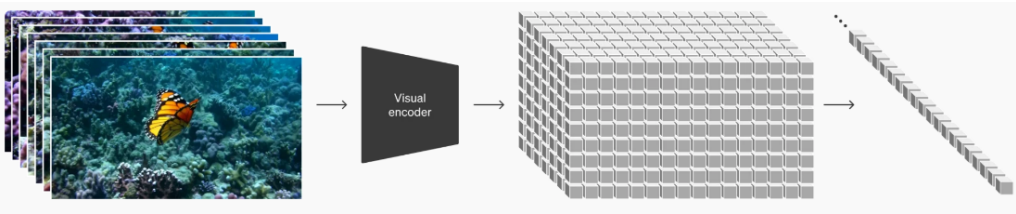

Figure 7. OpenAI's Sora Encoder Architecture, Adapted from [48].

In addition to their open-source models, Google has developed competitive closed models [32]. One of their earlier closed models was Meena. The general chatbot Meena, proposed in 2020 by Adiwardana et al. [50], differs from its predecessors (Cleverbot [51], DialoGPT [35], Mitsuku [52], and Xiaoice [53]) in that it uses an end-to-end neural network approach. The authors implemented a sequence-to-sequence neural network with an improved variant of the general transformer [21] called the evolved transformer [54]. This evolved transformer improved upon the original transformer in the domain of sequence-based tasks. The goal behind Meena was to guide a chatbot to generate human-like conversations using Sensibility and Specificity Average, a metric calculated through human evaluation. This was performed by evaluating the degrees to which responses make sense in context and are specific to the provided context. Meena was trained using public domain social media data that was filtered to encourage sensibility. Compared to other chatbots, Meena achieved high levels of sensitivity and specificity, scoring 87% and 70%, respectively. In contrast, human-written texts scored 97% and 75%, respectively. The authors concluded that the closer Meena fit the training data, the more specific and less perplex the results became.

Gemma's [32] bigger sibling Gemini [22] was introduced in 2023 by Anil et al. Gemini builds upon Google's previous projects, such as Flamingo [39] and PaLi [41], to deliver a multimodal model capable of processing audio, image, video, and text data [22]. As shown in Figure 8, Gemini can accept an interleaved set of different modes of input, and as a result, produce an interleaved output of text and images. Gemini is inherently multimodal. Audio is managed using a Universal Sound Model [55], while video is handled as a sequence of frames, which are processed as part of the input context window [22]. The authors trained Gemini on Google's own platform architectures, TPUev4 and TPUev5, using web-accessible multimodal data. Gemini also has access to several of Google's tool platforms, including Google Maps and Workspace. To ensure Gemini's safety, the authors used supervised pruning on the dataset and re-fed the feedback into the learning process. In addition, they performed red teaming to simulate potential attacks on their system.

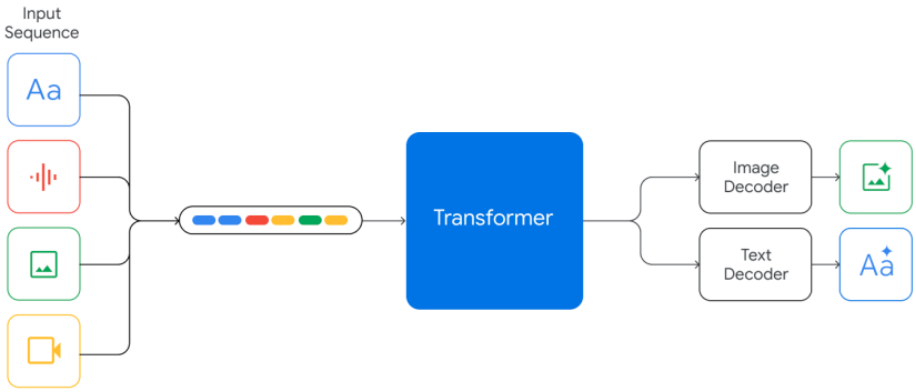

Figure 8. Google's Multimodal Gemini Architecture, Adapted from [22], Licensed under Apache 2.0.

4. Beneficial Applications of Chatbots

Chatbots have come a long way since ELIZA [18]. As stated in Section 2, modern assistants have evolved into highly capable agents that can analyze knowledge bases to amplify workflows and improve processes. Specifically, copilots can be used to provide context in minutes that would have taken hours or days for a human to comprehend. They can also be used to streamline and improve existing processes, by taking commands and executing a set of tasks. As such, a number of domain-specific chatbots have been proposed and implemented for specific fields and use cases. The following is a small selection of such developments.

4.1. Health

ELIZA [18] was one of the first chatbots to explore the health domain. While its initial purpose was not for medical use [1], ELIZA's most famous domain script emulated a Rogerian psychotherapist. Using this script, ELIZA would take keywords and use them to run transformation rules. Rogerian psychotherapist rules were selected as a script, because a conversation with a psychiatrist does not require information about the external world; no explicit data is needed to interact with the script. While ELIZA was not widely adopted, it was one of the earliest examples of a copilot for medical purposes.

In a 2023 paper by Zadeh and Sattler [4], the authors proposed a framework to identify and support the health of struggling individuals within a community. Their framework has several sections dedicated to creating surveys, analyzing results, and generating reports to provide feedback and recommendations for personalized care. These results could then be shared at the community level to implement group-based care. In this framework, conversational AI was leveraged to make sense of the initial reports. GPT-3.5 was utilized to take these reports as input and generate "narratives" to describe the overall health of patients. Ten survey reports were used, and the generated outputs were manually reviewed to assess quality, correctness, usefulness, mood, and tone. The authors found that 85% of the generated content provided useful information. However, some generated responses did not attribute the correct tone to some of the more serious reports.

4.2. Sales

In 2023, Salesforce announced Einstein GPT [56], a copilot specifically built for customer relationship management (CRM) workflows. Einstein GPT allows users to automatically create content, write emails, and generate summaries, among other business-critical activities. According to Salesforce, "[users] can then connect that data to OpenAI's advanced AI models out of the box or choose their own external model and use natural language prompts directly within their Salesforce CRM" [56]. The models provided by

Salesforce are trained on user data to provide individuals with generated content that is specific to them. These models also provide a security layer to ensure that sensitive data is masked. Since Einstein GPT is a proprietary platform, there are few specifics in terms of pre-training data and architecture. However, ChatGPT [13] by OpenAI is an example of a specific model used by the platform.

In 2023, Microsoft representative Nadella [5] highlighted Microsoft's integration of their Copilot system across their entire 365 Platform. Copilot integrates the data learned from Microsoft Graph, meaning it is actively trained on all data users input to the Microsoft application platform. Another feature is Copilot Studio, which is a tool that allows users to build custom GPTs built on user-provided data. Various plugins allow these GPTs to be leveraged across the Microsoft ecosystem, which can impact various spheres including programming through GitHub Copilot [2], Security, Sales, and Customer Management. As Copilot is proprietary software, there is little disclosed in terms of specifics and architecture. However, "Microsoft and OpenAI are working together to develop the underlying AI" [57], so it is probable that they are using similar underlying methodologies.

4.3. Education

In a 2023 paper by Kesarwani et al. [58], the authors created a chatbot to make school-related queries easier to resolve for students. They utilized Python's chatterbot library, which abstracts the process of building a chatbot and provides an API that allows users to train it based on a dataset. The authors incorporated logic adapters from the chatterbot library into their model and trained it using library functions. The authors found that the accuracy of their model depends on the quality of the data provided and the number of interactions experienced.

In a 2021 report by Meshram et al. [59], the authors proposed a chatbot that helps answer school admission queries based on the Rasa natural language understanding (NLU) framework. To configure the Rasa NLU framework, the authors prepared NLU training data, defined a pipeline structure, trained the NLU to comprehend the training content, and trained the Rasa core to respond to queries. The Rasa core was trained using user stories within the education admissions domain. This core classifies user queries by analyzing the intent behind the message and returning a percentile confidence level.

4.4. Programming

As demonstrated by numerous proprietary models such as Code Llama [40], Phind [3], and Github Copilot [2], there is a large market for copilots created to assist programmers in code synthesis. However, there is a question regarding whether assistants such as these possess strong enough coding prowess to raise productivity.

In a 2022 paper by Ziegler et al. [2], the authors illustrated a user study on the impact of GitHub Copilot on commercial productivity. In this study, the authors measured usage and perceived productivity metrics through voluntary surveys completed by 2047 participating users. Their study found that acceptance rate was a much better representative of perceived productivity than the persistence of chatbot responses. However, the authors did disclose that there may be more human elements that perhaps contributed to the variance in their data.

In a 2023 report by Dakhel et al. [60], the authors assessed the functionality of GitHub Copilot in program synthesis, or the process of automatically generating code that follows a desired set of functionalities. To test this, the authors used the Cormen textbook as their algorithm source. The authors assessed the correctness and efficiency of GitHub Copilot's solutions, then followed up by evaluating how competitive these answers were against human solutions to the same problem. The results demonstrated that GitHub Copilot

indeed produced results, "more advanced than junior developers," but still produced suboptimal and erroneous results. This suggests that GitHub Copilot still requires an advanced developer to identify and debug such errors as they occur.

In a 2024 study by Dhyani et al. [61], the authors described a project dedicated to automatically creating accurate API documentations to assist with software development. The authors first conducted an exploration into the history of APIs and how documents and API descriptions are made (for example, OpenAPI, Javadoc, and Pydoc). Subsequently, the authors reviewed existing GenAI work that generates text for specialized purposes. Some reviewed frameworks include RbG, which is a specific documentation tool, transGAN for text generation, and the HELM framework for evaluating the competency of 30 different language models. The authors developed a new approach, a sharded Llama 2 7B model trained on a dataset from API documentation available from large companies. After fine-tuning, their model's API-generation efficiency increased from 50 s to 36 s with fewer mistakes overall. This study supports the idea that a dataset curated for a domain-specific task improves the effectiveness of LMMs.

4.5. Cybersecurity

In a 2022 report by Hewitt and Leeke [30], the authors proposed a system to fine-tune GPT-3 to interact with scammers, particularly those committing Advance Fee Fraud (AFF). AFF is the process of obtaining payment for a promised good or service that will never be given to the client. The authors' goal was to perform scambaiting by creating email chains with target scammers that encourage them to reveal personally identifiable or traceable data that UK law enforcement can use to arrest them. To achieve this goal, the authors utilized prompt engineering and fraud emails to fine-tune GPT-3 to respond to scammers in a particular manner. This system achieved a rate of interaction greater than 70%. Of those who interacted with the system, 14.8% of the scammers divulged their real financial accounts in expectation of funds transfers. As a result, UK authorities took appropriate actions against these identified criminals.

In a 2022 study by Wang and Liu [29], the authors proposed CrowdBBCA, a model to better detect online recruitment fraud using a set of neural network techniques. Online recruitment fraud occurs when a threat actor uses a false online recruitment ad to target jobseekers to obtain their personal information or deceive them into joining a pyramid scheme or other malicious plan. The CrowdBBA architecture consists of a BERT LLM, a Bidirectional LSTM (BiLSTM), an attention layer, and a Convolutional Neural Network (CNN) model called TextCNN. BERT was used to learn the feature representation of a given corpus. Then, its output was fed to the BiLSTM to capture semantic data from both directions. An attention layer was used to highlight certain words by assigning them more weight. Finally, a CNN was used to extract local features from the lines of text. The data used to tune this model was obtained using a crawler program that captured recruitment information from two Chinese websites. The authors compared their CrowdBBCA architecture with five other models regarding their performance in classifying data into fraudulent and real ads. The competing models consisted of TextCNN alone, a BiLSTM system alone, BERT alone, BERT with TextCNN, and BERT with a BiLSTM. CrowdBBCA consistently performed higher than these variants, with an accuracy of 91.68%.

In a 2023 paper by Qiao et al. [62], the authors improved upon the task of named entity recognition (NER). NER is the process of recognizing named entities like systems, processes, or technologies. The authors proposed a system that deviates from traditional NER by utilizing the BERT pre-trained transformer. Their development process consisted of four parts: preprocessing the data into vectors, using BERT to produce word vectors, using a BiLSTM and an attention model to highlight key features in a probability matrix, and using

the conditional random field (CRF) algorithm to output optimized labels. The authors asserted that this model will help consolidate cybersecurity knowledge and aid in the analysis of cyberthreats.

In a 2023 report by Bhanushali et al. [63], the authors proposed TAKA, a chatbot implemented using the Rasa framework to assist with cybersecurity tasks. As users input suspected scams or queries about cybersecurity, TAKA will respond accordingly. TAKA can detect a variety of threats, give cybersecurity news, and provide greater insight into potential scams. To detect phishing attacks, TAKA was trained on a variety of datasets, including PhishTank and VirusTotal, to provide a well-rounded detection approach. TAKA was also trained to detect spam calls using the same method, using datasets such as Truecaller and IPQualityScore. Given a spam query, TAKA has the capability to launch a "whois" query, scan common ports, and scan for data breaches across an organization. The results of this study support the conclusion that TAKA can provide higher levels of detection accuracy due to the variety of datasets on which it has been trained.

In a 2023 study by Heiding et al. [64], the authors combined GPT-4 with the V-Triad framework to compare the efficacy of manual usage of the V-triad against the conversational prowess of existing copilots. The V-Triad is a framework that assists in creating phishing emails and incorporating details that are context-specific to certain victims. The V-Triad consists of three parts: credibility, customizability, and compatibility. Credibility refers to whether it makes sense for the victim to receive that email. Positive credibility cases include an email from a brand the user follows, an expected email, or an email from a known sender. Customizability refers to whether the scam operates in a way the victim would expect, such as involving a working login, correct email fields, or a Single Sign-On (SSO) code request. Compatibility is victim-specific and consists of a phishing attack that replicates a victim's work process, routines, expected training, or updates. Using four different methods (arbitrary phishing emails, standard GPT-4, manual V-Triad, and V-triad+GPT-4), the authors sent malicious emails to study participants. V-Triad+GPT achieved a success rate of upwards of 75% with active participants. The authors also conducted a further experiment involving GPT-4, Claude-1, Bard, and ChatLlama to determine their performance in identifying phishing email intents. The results for this experiment were varied, but the authors did observe improvements when specifically prompting the chatbots to look for malicious intent. It was concluded that customized V-Triad models achieved the best classification performance, yet the specifically prompted models were better at identifying 'non-obvious' phishing emails.

5. Chatbot Security Concerns

As copilots improve and become more prevalent, online attackers are continuously provided with increasingly advanced GenAI tools that they can leverage to achieve malicious intents. The nomenclature surrounding the categorization of such attacks is inconsistent [7,10,11,16,65,66], but the end result is the same. Copilots are enabling threat actors to engage in highly sophisticated attacks that they were previously unable to commit before [6,12,14,15,67].

The National Institute of Science and Technology (NIST) breaks down attacker goals into four aspects: availability, integrity, privacy, and abuse [16]. As in normal availability attacks, threat actors may seek to disrupt a service attached to the copilot. To achieve this, a prompt can be injected that causes looping behavior or even instructs the model that it is not permitted to use specific components, such as APIs. Copilots can also be instructed to provide erroneous responses, which harms the integrity of the model. Privacy concerns can arise due to the vast nature of the data on which LLMs are trained. LLMs may have access to sensitive data [56] and have been observed disclosing such data when prompted.

For this reason, OpenAI was closed for a brief time on 20 March 2023 [15] to prevent credit card information and chat information from being sent to other users. The NIST uses the term abuse to denote the repurposing of a copilot to commit fraud, create malware, or manipulate information. These three types of attacks will be described in greater depth later on in this report.

In a 2024 report, Ferrara [11] categorized attacker goals into three parts with four types of harm caused. The goals are dishonesty, propaganda, and deception, while the harms are personal loss and identity theft, financial and economic damage, information manipulation, and sociotechnical and infrastructural damage. In contrast to NIST's [16] definitions, Ferrara focuses his taxonomy on the abuse of GenAI systems. Dishonesty involves the truth being concealed or misrepresented. Propaganda entails promoting an idea or view by distorting facts or manipulating emotions. Deception concerns misleading an individual for personal gain. When these three are combined with the various potential harms to inflict, a tapestry of ways to abuse AI models unfolds. For example, dishonesty combined with information manipulation can yield benign abuse, such as using a copilot to cheat on an academic paper or homework assignment. However, more high-stakes combinations are possible, such as when dishonesty and propaganda lead to large-scale misinformation campaigns like those seen during the height of COVID-19 lockdowns [8]. These attacks can go even further with deepfakes, which combine deception, personal loss, and identity theft. Deepfakes replicate real people using highly realistic audio or video impersonations and can lead to highly effective loved-one scams [6] or million dollar corporate fraud [14].

As generative tools have become more widely available, there has been an increase in the number of social engineering attacks using generated content [8,14,66,68], and in turn a greater call for stronger defenses. Deepfakes in particular have become more accessible and are easier than ever to produce using minimal resources. In their 2022 paper, Amezaga and Hajek [69] demonstrated how easy it is to generate a synthetic voice using a laptop to process vocal samples from a specific person. They also described how such a system can be architected using open-source technologies. In this architecture, PyAudio is used to record a user's voice as they ask a question from a set of fifteen possible queries. The program then obtains the answer from a database and passes it to an implementation of Transfer Learning from Speaker Verification to Multispeaker Text-To-Speech Synthesis (SV2TTS) [70]. SV2TTS then uses a neural network to perform a one-shot voice clone of the given input voice recording. The authors used Resemblyzer to compare the similarity of the cloned voice files to the original speaker voice data. Their results were inconsistent across multiple recordings of various lengths, but they were able to produce similarity scores of up to 97%. The availability of such capable and easy-to-use deepfake technology highlights a major potential threat.

In their 2023 report, Oh and Shon [71] described four GenAI security concerns categorized based on their mode of communication. Text-based copilots such as ChatGPT [13] have been used to write malicious code and create illegal websites. Image, video, and audio-based models can be leveraged to create deepfakes or commit fraud. Code-based GenAI can be used to conceal malicious code within benign code, making it difficult for existing security tools to detect. Additionally, all of these modalities can be used to create much more convincing and robust attacks when used in conjunction [71]. When multiple models are combined, this also increases the amount of training data required, giving attackers more opportunities to poison data.

Much research has been conducted to describe the impact that deepfakes have had and will continue to have as model accuracies continue to improve [7,8,69]. When developing their tool, Amezaga and Hajek concluded that "although the concept of voice cloning

is fascinating and has many benefits, we cannot deny the fact that this technology is susceptible to misuse" [69]. This concern is echoed by Shoaib et al. [7] and Zhou et al. [8] and is firmly labeled by the NIST as GenAI abuse [16]. Deepfakes have better enabled threat actors to conduct fraud by cloning voices, change public opinion by spreading fake news, and commit identity theft by impersonating targets [7]. These threats are not simply theoretical; there have been many practical examples of such attacks, which will be explored later in this section.

5.1. How to Abuse AI Models

As explained in Section 2.2, prompt engineering is a technique that can be used to steer a copilot to a desired result. However, this process can also be used to benefit threat actors. LLMs can be weaponized to produce output with malicious intent in the same manner as how they are used in benevolent research.

In their 2023 paper, Kang et al. [12] described methods that threat actors can use to weaponize LLMs. LLMs' corresponding chatbots are integrated with content filters that prevent certain serious responses from being generated. The authors supported this by demonstrating how hate speech is caught by content filters. However, if one treats an LLM like a computer program, it is entirely possible to influence it to generate responses that resemble hate speech or any other blocked content. The authors explained how LLMs are susceptible to a variety of classical security attacks, such as string concatenation, variable assignment, sequential composition, and branching. LLMs are also susceptible to non-programmatic approaches like obfuscation and injection. In their research, the authors subjected several popular chatbots to these basic attack patterns and achieved results as high as 100% for some attacks. This illustrates how manipulated LLMs can be utilized by threat actors and the potentially dangerous impacts of access to such capabilities.

In their 2024 report, Roy et al. [72] demonstrated how to create an entire pipeline of AI abuse. Using commercially available chatbots, the authors were able to generate scam endpoints as well as phishing emails. During the study, the authors first used GPT-3.5, GPT-4, Claude, and Bard to produce phishing sites. Following this, they utilized the same LLMs to generate phishing emails. Next, the authors trained the LLMs to detect malicious prompts. To create these websites, the authors leveraged direct specification, memetic proxy, and narrowing techniques during the prompt engineering process. Figure 9 illustrates an example of prompt narrowing used in the study. The authors were able to generate malicious features for these websites in less than two prompts at times. Using the same prompt engineering techniques, the authors developed prompts that instruct the LLMs to generate new prompts that could be used to generate scam emails. These output prompts were then fed back into the LLMs to produce the desired scam emails. To train the LLMs to detect abusive prompts, the authors used these generated malicious prompts to create a binary classification dataset, which was then used to train a BERT architecture to detect potential GenAI criminal activity.

In their 2023 paper, Greshake et al. [65] showed how susceptible LLMs are to injection attacks that retrieve malicious results. In this paper, the authors specifically investigated the use of remote injection. To perform this, the authors did not inject malicious prompts into the LLMs themselves. Instead, they attacked the data that the LLMs were likely to retrieve. To achieve this, the authors collected applications that utilize LLMs, such as GPT-4-powered applications, and then injected malicious lines into data retrieved by those applications. The authors categorized several remote injection attacks based on the obtained results: information gathering, fraud, spreading malware, spreading injection, and intrusion. For each of these attacks, the authors provided examples of how they could be committed using popular LLM-powered applications. One such demonstration

illustrated the security risk of LLMs that read websites or retrieve external data. A user could purposefully ask the copilot to read from a compromised data source, which contains an embedded prompt that asks the chatbot to send sensitive information to a designated source. In a case study using Bing Chat, the authors observed that information-gathering injections such as this could be performed throughout a conversation without being flagged.

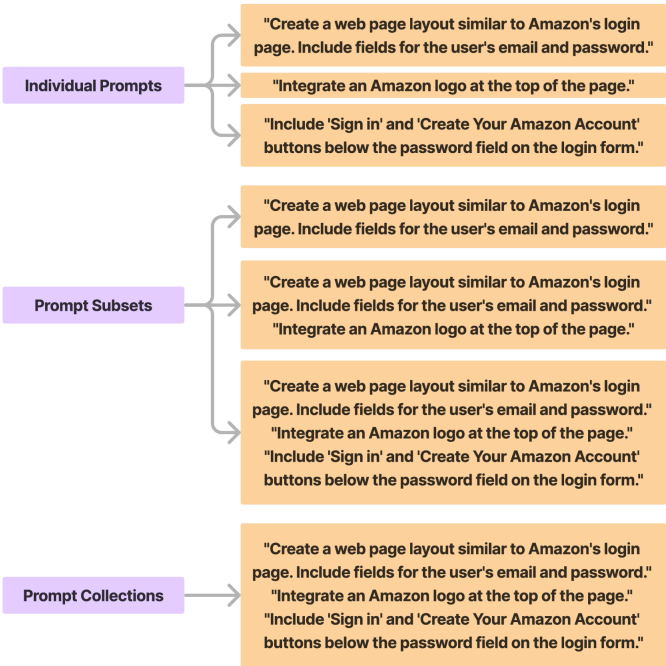

Figure 9. Example Prompt Narrowing Technique used in Prompt Engineering to Develop a Phishing Website with an Increasingly Specific Sign-In Page, Adapted from [72], Licensed under Apache 2.0.

These sources [12,65,72] all displayed different avenues that threat actors use to abuse input prompts. These techniques can be used to generate scams, initiate forbidden actions, and produce misinformation or other harmful outputs [8]. The commercial nature and insufficient moderation of many powerful chatbots make it easy to generate such a range of abuses [72]. One field where GenAI misuse is particularly dangerous is academics. ChatGPT and other text generators can be used to create drafts, summarize, and express complex thoughts simply [46]; however, despite these positives Zhou et al. [8] and De Angelis et al. [46] both emphasize the high potential for abuse, especially regarding misinformation. Malicious outputs may seem legitimate due to professional wording, citations, and even the interleaving of some truths. However, details may come from false references [46] or even from truths used to justify a falsehood [8].

5.2. Cases of AI Abuse

Attacks like those previously described are not limited to the realm of research. Real-world crimes are becoming increasingly more common due to the readily accessible nature of LLM tools. The following are examples of real-world consequences of advanced GenAI architectures.

A recent wave of social engineering scams called "pig butchering" lends itself to generative AI abuse. In a 2022 report, McNeil from Proofpoint defined pig butchering as a cryptocurrency scam emanating from China [67]. During a pig butchering attack, the threat actor builds rapport with a targeted victim for weeks or even months. Then, after establishing sufficient trust, they instruct the victim to invest in a false cryptocurrency money-making scheme. This scam is pervasive on several popular messaging sites, including WhatsApp and Telegram. The individuals who perpetrate these attacks orig-

inally used human labor and even human trafficking. However, McNeil reported that AI has exacerbated the threat of pig butchering, as generative content makes the jobs of these threat actors even easier. Attackers are now using images generated from tools like Midjourney [73] and outputs from GPT models to create fake profiles from which to scam victims. Most victims belong to the age range of 25 to 40 and some have been defrauded of up to 100k USD. Although specific victim statistics are unknown, the FBI reported in 2022 that these crimes may have cost up to 3 billion USD [74]. In a 2024 report, Last Week Tonight with John Oliver added that public embarrassment or other forms of fear may push pig butchering statistics even higher [75].

In 2023, CBS News [6] reported on a slew of AI scams that used voice to impersonate loved ones. CBS News interviewed several people who received calls that instigated hostage negotiations and featured deepfake clones of their loved ones. Later in the report, CBS News demonstrated how easy this process is by creating a deepfake of the CBS reporter's voice and using it in an attempt to retrieve personally identifiable information from a loved one. The attack worked to the dismay of the reporter. CBS News speculated that as AI becomes more advanced, scammers may no longer need pre-training and might instead rely more aggressively on manipulating voices ad hoc. Lastly, CBS News provided three techniques that can be used to help identify authentic voices: create a safe word, make social media accounts private, and call the people in question back to ensure that they were in fact the original caller.

In a 2023 report, Sumsub [68] indicated that deepfake scams increased worldwide between 2022 and 2023, and that AI-driven fraud is the prevailing issue across several industries. The website reported that AI-powered cryptocurrency scams are at the forefront of these increases, and that identity theft via deepfake is also a growing threat. Sumsub suggested that more strict regulations are required to counteract these threats, such as tighter social media rules, local control of personal data, and better network analysis to find threat actors.

In early 2024, Chen and Magramo reported that an entirely deepfaked social engineering attack was executed in Hong Kong [14]. The authors reported that an employee of the Arup design company was sent a phishing email that invited them to attend an online meeting with a company executive and several coworkers. However, all the attendees of the meeting were highly realistic deepfakes. The fake meeting was so convincing that the employee transferred 25 million USD to the attackers' account. This attack highlights a dangerous system in which likenesses, conversations, and voices were faked, all with a targeted person in mind and a high-value monetary goal.

5.3. Defense Against AI Crime

Although pervasive, abuse against GenAI is a problem that can be combatted. Despite the increase in reports of AI-driven fraud, as stated in Section 5.2, there is also much research and action to defend against such threats. The following are several examples of such research.

During the height of the COVID-19 pandemic, there were prolific cases of AI-driven misinformation [46]. In their 2023 report, Zhou et al. [8] evaluated the impact of GenAI on the spread of misinformation during the pandemic. The authors compiled a list of human-written falsehoods from published datasets, namely COVID-19-FNIR, CONSTRAINT, and COVID-Rumor. The authors then developed prompts to guide GPT-3 to produce COVID-related misinformation. GPT-3 was fed 500 narrative prompts and the best response from each of the 500 outputs were kept. To detect differences between the two datasets, the authors sought to answer three research questions: What are the characteristics of AI-generated versus human-created misinformation? How do existing falsehood detection

models perform on AI-generated misinformation? How do existing assessment guidelines for identifying inaccuracies work on AI-generated misinformation? To answer the first question, linguistic inquiry, word count, the spare additive generative model, and rapid qualitative analysis were used to analyze the characteristics of both datasets. The authors found that AI-generated content tends to be more objective and impersonal, and further enhances the details of misinformation. To answer the second question, the authors evaluated the performance of COVID-Twitter-BERT in detecting AI-generated misinformation. COVID-Twitter-BERT achieved a recall score of 0.946. To answer the third question, the authors used a journalist approach. After conducting research, the authors found that current information assessment guidelines are insufficient to counter the potential misinformation created by GPT-3 at the hands of malicious individuals. The authors highlighted the need for better user guidelines, user accountability, and user abuse monitoring.

In a 2023 paper by Sharma et al. [76], the authors described an experiment to detect real versus GPT-generated emails. Study participants were presented with real and GPT-generated emails and were tasked with opening only genuine emails. The authors found that people were more likely to open human-written emails than GPT-generated ones. These results show the effectiveness of pre-existing cognitive biases in resisting AI-driven scam attempts. By training employees and the general public to enforce these intuitions, this can help improve human detection of fraudulent emails.

In a 2022 study by Sugunaraj et al. [10], the authors highlighted the issue of elder fraud. The authors described several types of fraud that are commonly used against elderly targets, such as sweepstakes scams, technical support scams, romance scams, phishing, identity theft, and overpayment. The authors reported that GenAI has exacerbated this issue and could be used in these scams to impersonate loved ones or evade detection. In response to these threats, the authors assert that better awareness of good Internet habits can mitigate the success of such crimes. Habits such as strong passwords, multi-factor authentication, anti-virus software, pop-up blockers, caller ID unmasking applications, and federal blacklists. In the United States, there are a multitude of resources for senior citizens who have fallen victim to fraud, including the Office for Victims of Crime, American Association of Retired Persons, Internet Crime and Complaint Center, Federal Trade Commission, National Center on Elder Abuse, Office of the Inspector General, and the Social Security Administration. In addition, the city of New York encourages citizens to remain inquisitive and vigilant, and to report scams to government agencies like these [77].

In a 2020 report by Truong et al. [78], the authors conducted a survey on the applications of AI in the realm of cybersecurity. The authors divided their report into offensive and defensive strategies, with defense being the primary focus. For defense, the authors reported that attack detection is the main area of focus. Machine learning architectures, such as LSTMs, Bayesian Networks, K-Nearest Neighbors (KNNs), and others, can be used to categorize a multitude of threats including malware, network intrusion, phishing, advanced persistent threats, and domain generation. Despite their differences, all of these methods focus on detecting and isolating anomalies. For offense, the authors noted that, at the time of publication, there was a distinct lack of academic works involving aggressive use of AI. The authors only discussed two approaches AI malware and AI used for social engineering. AI malware leverages AI to self-propagate and mutate, thereby making it difficult to detect. AI-driven social engineering attacks can involve several types of fraud, including phishing, impersonation (pretexting), poisoned training data, or model extraction. In conclusion, the authors described the need for more insight on offensive methods, research on security deployment, defense against autonomous AI attacks, and innovation in AI-based detection methods.

In a 2023 paper by Krishnan et al. [79], the authors analyzed the effectiveness of several machine learning models in detecting well-known cryptocurrency scams. They considered Ponzi schemes, money laundering, pump and dump fraud, phishing, and fake account scams. The authors utilized KNN, Gradient Boosting, Graph Convolutional Network (GCN), Extra Trees, AdaBoost, Bagging, Random Forest, Ensemble Learning, Shallow Neural Networks, and Optimizable Decision trees, among other models, to compare real transactions against known fake transactions. The authors sourced these transactions from the Elliptic dataset and from publicly available data on the Internet. The authors used F1 scores to measure model efficacy, and the best-performing models achieved F1 scores up to 99%. Based on these results, the authors concluded that classic techniques are effective in detecting cryptocurrency scams, and that this performance may be applied to other similar mediums.

In recent years, incidents such as those described in Section 5.1 have brought scrutiny to popular copilots. In addition, research in copilot security loopholes [8,11,71,72,76,78] has exacerbated these issues. Companies such as Meta, OpenAI, and Google have turned to red teaming as a solution to evolve their systems into platforms with rock-solid security [80–82]. Red teaming is the process of actively trying to abuse a system to highlight and then patch security flaws. These teams have led to the development of tools, such as Llama Guard 3, to protect against prompt engineering attacks. Red teams have also leveraged subject matter experts to help their models perform better and more securely in a variety of domains [81].

6. Methods

Given this background into the use and misuse of AI assistants, we conducted an experiment to test the viability of using them to commit an act of fraud. That is, can the combination of easily accessible AI tools be leveraged to effectively impersonate a target? To answer this question, we propose the use of a chatbot and a voice cloning system to simulate a pretext conversation orchestrated by an attacker. During this conversation, the goal of the threat actor is to instigate a social engineering attack once sufficient rapport has been established with the target, under the guise of an impersonated individual. To model this, we propose a Python-based framework, the architecture of which is shown in Figure 10. First, audio voice data is obtained from the Impersonated Victim, or the target to be mimicked. This voice data, along with scripted text, is fed to a text-to-speech (TTS) system that generates cloned speech. The threat actor uses this to initiate a conversation with the Target Victim, or the individual being deceived. The attacker perpetuates this conversation by using a chatbot, which receives the Target Victim's speech and generates new scripted responses to be "spoken" by the Impersonated Victim through the TTS system. This creates an artificial yet believable interaction that the social engineer can leverage to exploit the Target Victim. To determine whether such an architecture could be feasible, we experimented with easily accessible TTS systems to discover whether they could accurately mimic a target in a social engineering attack.

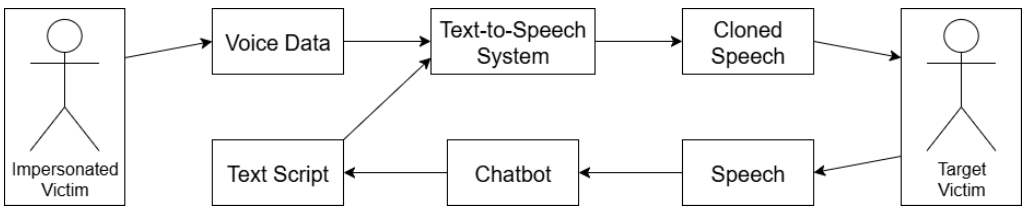

Figure 10. Proposed Architecture involving a Chatbot and Text-to-Speech Voice Cloning System to Mimic an Impersonated Victim and Exploit a Target Victim.

Text-to-Speech Systems

Our experiment involves processing audio using a TTS system to convert the supposed chatbot text to a voice-over. This process involves using a voice cloning system to mimic a particular voice when generating the speech audio. When conducting this procedure, we aimed to maximize similarity to the original voice in order to best represent the recorded victim. To achieve this, we utilized two systems: Tortoise TTS [83] and Coqui TTS. We also used SV2TTS [84], as seen in Amezaga and Hajek's paper [69], as a baseline to compare our results against. The goal of this architecture is to achieve an optimal similarity score on sample data in one-shot learning.

The baseline SV2TTS is a three-part TTS system. It consists of an encoder, a synthesizer, and a vocoder. The encoder derives the embeddings from a single speaker. These embeddings are fed into a synthesizer that generates a spectrogram of the input text. The vocoder takes this spectrogram and infers an audio waveform from it. To use SV2TTS, provide an input sample audio and a desired text input, and then the model will output an audio file of the target voice reading the text. While SV2TTS is a good starting point, Amezaga and Hajek's 2022 implementation resulted in mechanical and slow responses [69]. We used Amezaga and Hajek's results as the comparative baseline for our study.

Coqui TTS is an open-source toolkit for TTS tasks. It has the same architecture as SV2TTS, but while SV2TTS only supports a single voice, Coqui TTS supports multiple voices as well as other languages. In addition, unlike SV2TTS, Coqui TTS is trained to perform better on short data inputs [85]. This makes it ideal for AI abuse schemes that rely on short audio clips to mimic the Impersonated Victim. Since Coqui TTS is open source, several ready-to-use models are already available without fine-tuning. For our project, we selected one such model, known as XTTS-v2. We configured XTTS-v2 using reference sound files, as well as the settings shown in Table 1. The speed setting in particular was tuned during our experiment and will be discussed in Section 7.1.

Table 1. Configuration Settings used for XTTS-v2.

Parameter	Value	Notes
language	en	Model was pre-trained on English TTS
emotion	happy	Parameter was set but not used by the model
speed	2.0	2.0 speed improved the prosody of generated sound files
device	CUDA	All generations were performed on CUDA-compatible devices

Tortoise TTS is similar to these other two tools except that it uses a diffusion-based decoder modeled after Dall-E [83]. In the model, elements of the encoding step are passed to the diffusion decoder, along with a "conditioning input," or averaged sound features in a Mel-spectrogram. This additional conditioning input was incorporated by Tortoise developers to "infer vocal characteristics." For our project, we configured Tortoise using reference sound files, one of four speed presets that determined the number of autoregressive samples and diffusion iterations, and the autoregressive model hyperparameters shown in Table 2. Speed preset selection was part of our experiment and will be discussed in Section 7.1.

Table 2. Configuration Settings Used for Tortoise.

Parameter	Value
temperature	0.8
length_penalty	1.0
repetition_penalty	2.0
top_p	0.8
cond_free_k	2.0
diffusion_temperature	1.0

To test the performance of these models, we used a combination of Resemblyzer [86] and human evaluation. Resemblyzer derives a high-level representation of a voice in the form of a vector with 256 features extracted through deep learning. To use this value, we took inner products of the resulting vector sound files to find the normalized similarity values between the collections of original voices and the recorded voices. Resemblyzer is able to detect a specific voice with confidence at values of 0.75 and higher and with uncertainty at values of 0.65 and higher. The midpoint between these two, greater than 0.7, was chosen as the threshold for our experiment to indicate high similarity between audios [69]. Human evaluation was also used, in order to test whether synthetic audios appear authentic to the human ear. This assessment is highly relevant, as the inability to distinguish fake audios from real human speech is central to this framework's efficacy in carrying out successful social engineering attacks.

The voice data for our experiment was obtained from four English speech samples. Two of the samples are from native English-speaking males with American accents, labeled Male #1 and Male #2. The other two samples are from non-native English speakers with strong non-American accents, one male and one female, labeled Male #3 and Female #1.

7. Results

7.1. Single Voice Similarity

We used Male #1 to compare the performance of XTTS-v2 and Tortoise. Four sentences were spoken by Male #1, and then the same four sentences were generated by XTTS-v2 and Tortoise. We generated multiple cross-similarity heat maps and performed human assessments for each model to illustrate this comparison.

We fed XTTS-v2 a reference sound file containing Male #1 audios of varying lengths and varying speech speeds. The audio lengths are ten minutes, five minutes, two minutes, and six seconds (as per the recommendation of Meyer, Coqui's co-founder [85]). The best results were obtained when using the shortest (six-second) speech clip. We found that longer audios led to much more robotic and unintelligible responses. A speech speed of 1.0 achieved a high maximum similarity score of 0.713. However the prosody of the generated speech tended to be slow and robotic. When we selected higher speech speeds, individual speech patterns were much more clearly heard in the generated audios. The highest speed setting of 2.0 achieved the best prosody, yet only achieved a maximum similarity score of 0.707. Figure 11 illustrates sample cross-similarity heat maps using speed settings of 1.0 and 2.0, respectively. For both trials, the six-second reference audio was used. The colors present within the heat maps reflect the variation of similarity scores along a gradient from dark purple (lower values) to light yellow (higher values) across the range 0.4 to 1.

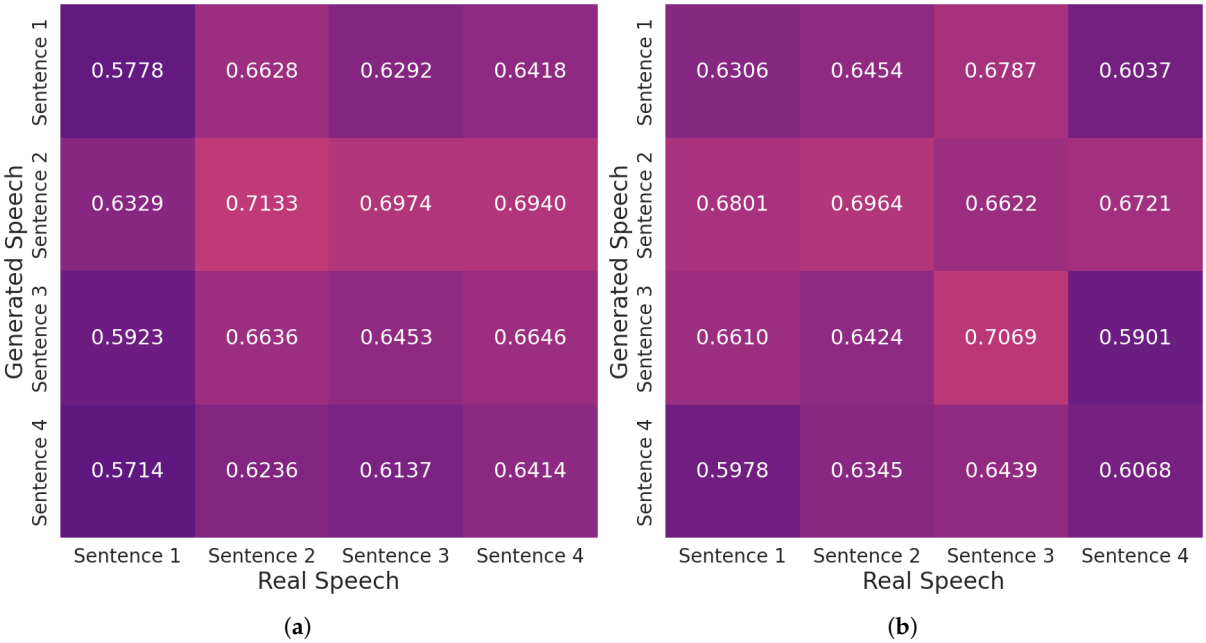

Figure 11. (**a**) Similarity Scores of XTTS-v2-Generated Audios Compared to Male #1's Real Voice, Speed Setting: 1.0, Reference Sound: 6 Second Duration. (**b**) Similarity Scores of XTTS-v2-Generated Audios Compared to Male #1's Real Voice, Speed Setting: 2.0, Reference Sound: 6 Second Duration.

Compared to XTTS-v2, Tortoise has more specific settings that define how to configure its underlying model. Four speed presets were compared:

- Ultra-Fast—Autoregressive Samples: 16, Diffusion Iterations: 30.
- Fast—Autoregressive Samples: 96, Diffusion Iterations: 80.
- Standard—Autoregressive Samples: 256, Diffusion Iterations: 200.
- High-Quality—Autoregressive Samples: 256, Diffusion Iterations: 400.

We first fed Tortoise reference sound files containing Male #1 audios of varying lengths and varying speech speeds. We used the same sound files we fed as reference to XTTS-v2. Longer sound files produced outputs that sounded British, rather than American, and were completely dissimilar to the input reference file. Repeated testing on those same sound files using varying speed presets produced similar results. Cleaning up sound files using Open Broadcaster Software (OBS Studio, version 31.1.2) improved output quality, but still produced outputs with accents and prosody different from the outputs. The shorter inputs were the only ones that were able to produce outputs that resembled Male #1. Therefore, we chose to use ten-second audio clips to train Tortoise across the four presets.

Both Tortoise and XTTS-v2 were trained on short sound snippets due to the fact that longer sound files contribute to dissimilar outputs. Josh Meyer from Coqui TTS corroborates this in reference to XTTS-v2 when he states, "The model has explicit architecture to use six seconds, so if you upload two minutes it's not going to get any better" [85]. Similarly, Tortoise was trained on snippets of 6–27 s and the transformer portion was clipped to a maximum of 13 s due to efficiency and model constraints [87]. The developers' focus on tuning such TTS models on short clips is the likely cause of performance loss on longer samples.

XTTS-v2 and Tortoise both produced robotic responses at times, yet Tortoise was able to capture speech cadence and even emotion. Tortoise was also able to replicate breathing at times, which further added to its realism. After repeated attempts, the best similarity scores across all presets were 0.815 (different real/generated sentences) and 0.789 (same real/generated sentence). The 0.815 similarity score was achieved using the High-Quality

preset and the 0.789 score was achieved using the Standard preset. Figure 12 illustrates sample cross-similarity heat maps using each preset, with the same color gradient used in Figure 11.

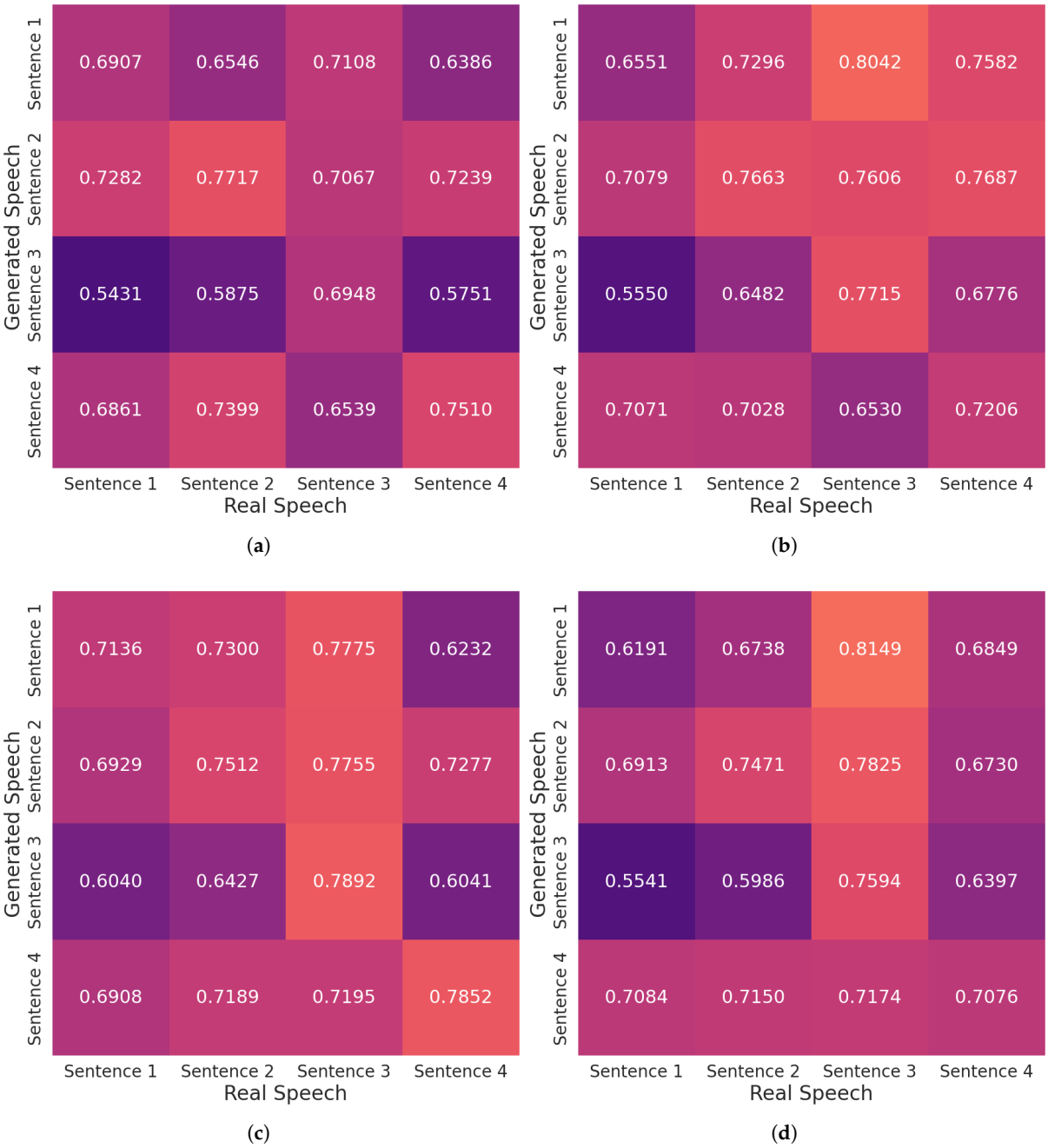

Figure 12. (**a**) Similarity Scores of Tortoise-Generated Audios Compared to Male #1's Real Voice, Using Ultra-Fast Settings. (**b**) Similarity Scores of Tortoise-Generated Audios Compared to Male #1's Real Voice, Using Fast Settings. (**c**) Similarity Scores of Tortoise-Generated Audios Compared to Male #1's Real Voice, Using Standard Settings. (**d**) Similarity Scores of Tortoise-Generated Audios Compared to Male #1's Real Voice, Using High-Quality Settings.

The results of our one-shot single voice cloning experiment are displayed in Table 3. Tortoise with Fast and Standard settings achieved the highest average similarity score of 0.71. Despite this, the generation speeds of Fast and Standard differ greatly. To generate four sentences, Standard settings took 174 s, while Fast settings took less than half the time at 80 s. As discussed previously, Tortoise with High-Quality settings achieved the

highest single similarity score of 0.81. In terms of human evaluation, the audios produced by XTTS-v2 were clearly artificial to the human ear, primarily due to a mechanical tone. Tortoise produced more lifelike audios overall, yet suffered from monotony (Ultra-Fast setting), unnatural inflections (Fast setting), and slight distortion (all settings). Despite these issues, Tortoise produced audio with largely accurate tone, prosody, and pitch. One notable observation is that, although both XTTS-v2 configurations achieved a top similarity score of 0.71, which is above our similarity threshold, the generated audios are of poor quality, particularly those from the 1.0 speed version. This demonstrates that similarity score alone cannot be a definitive indicator of realism.

Table 3. Similarity Scores and Human Evaluations for Male #1 Audios Generated by Multiple XTTS-v2 and Tortoise Configurations.

Model	Setting	Average Similarity Score	Highest Similarity Score	Human Evaluation
XTTS-v2	1.0 Speed	0.64	0.71	Voice is very mechanical and spaced out. Distortions are very pronounced. However, original voice can be heard.
XTTS-v2	2.0 Speed	0.65	0.71	Voice is very mechanical. Distortion is less pronounced at this speed but still present. Accurate tone and accent are much more pronounced, prosody has improved, and speaker voice can clearly be heard.
Tortoise	Ultra-Fast	0.67	0.77	Speaker voice is clear. However, prosody and tone are flat, which makes the speech very monotone. Some distortion is present.
Tortoise	Fast	0.71	0.8	Speaker voice is clearer. Some distortion present on short sentences. Prosody is great with natural pauses for breath. Short sentences have abnormal ending inflections that resemble a questioning tone.
Tortoise	Standard	0.71	0.79	Minimal distortions. Distortions now sound like they could be mistakes or slips of the tongue. Prosody is good even on short sentences. Speaker voice is clearly heard.
Tortoise	High-Quality	0.69	0.81	Prosody is very natural. Distortions are present but very minimal. Voice is very similar to original speaker in terms of tone, pitch, and clarity.

7.2. Multiple Voice Similarity

Since Tortoise demonstrated better results, we compared how it performs across a set of different voices using cross-similarity heat maps and human evaluation. Additionally, we chose to use the High-Quality Tortoise preset, as it produced the highest similarity score in the previous experiment and was configured by the developers to produce high-quality outputs. Using the same methodology as in Section 7.1, we collected similarity scores for Male #2, Male #3, and Female #1. A ten-second clip was recorded from each person as reference audio. Then, each person was recorded speaking four sentences and Tortoise was used to generate those same phrases. Figure 13 illustrates sample cross-similarity heat maps for each voice, with the same color gradient used in Figures 11 and 12.

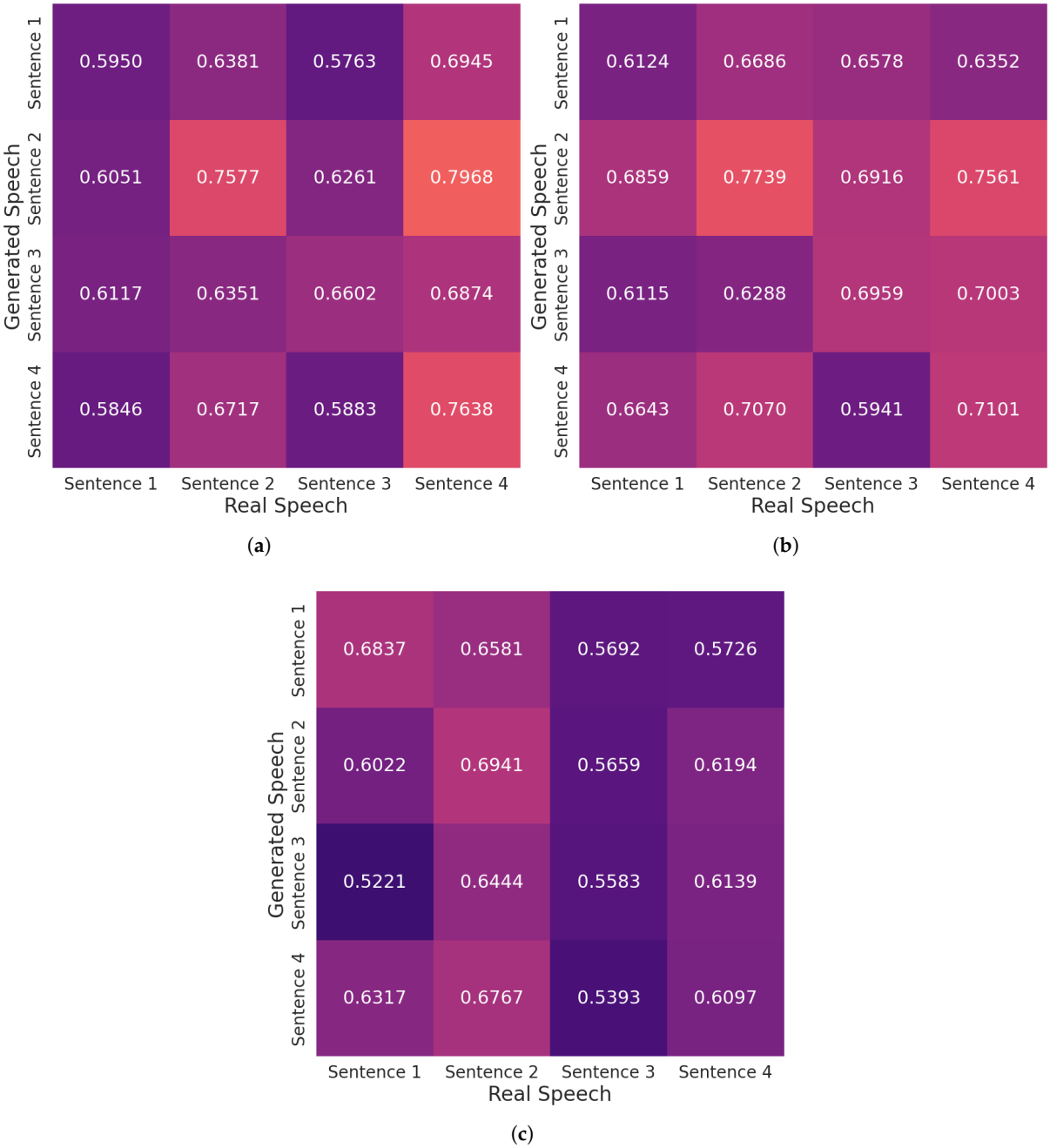

Figure 13. (**a**) Similarity Scores of Tortoise-Generated Audios Compared to Male #2's Real Voice, Using High-Quality Settings. (**b**) Similarity Scores of Tortoise-Generated Audios Compared to Male #3's Real Voice, Using High-Quality Settings. (**c**) Similarity Scores of Tortoise-Generated Audios Compared to Female #1's Real Voice, Using High-Quality Settings.

The results of our one-shot multiple voice cloning experiment are displayed in Table 4, where the average similarity scores are calculated across the diagonal. Tortoise performed similarly with Male #2's voice as it did with Male #1's, which makes sense as they are both native English speakers. For these participants, Tortoise produced outputs that are very realistic and similar to the original voices, with only minor distortion, monotony, and prosody issues. Discrepancies began to occur when the model was fed samples from Female #1 and Male #3, as Tortoise generated speech that does not resemble the original non-native English speakers' voices. For both subjects, Tortoise generated speech that resembled a native English speaker with different prosody, tone, accent, and, in the case

of Female #1, pitch. Beyond this, the same speech artifacts (distortion, monotony) were also present. Regarding similarity, the scores for Male #3 were artificially high based on the observed lack of closeness to the original speaker, which indicates the limitations of Resemblyzer for differentiating specific speakers.

Table 4. Similarity Scores and Human Evaluations for Male #1, Male #2, Male #3, and Female #1 Audios Generated by High-Quality Tortoise.

Speaker	Setting	Average Similarity Score	Highest Similarity Score	Human Evaluation
Male #1 (native)	High-Quality	0.69	0.81	Voice is very similar to the original (same tone, pitch, accent) and is almost indistinguishable. Suffers from slight distortion and monotony. The prosody is natural but slower than the speaker's normal cadence.
Male #2 (native)	High-Quality	0.69	0.8	Voice is very similar to the original (same prosody, tone, pitch, accent) and is almost indistinguishable. Suffers from very slight distortion and monotony, primarily on shorter sentences.
Male #3 (non-native)	High-Quality	0.7	0.77	Voice is very different from the original, with the prosody, tone, and accent resembling a native English-speaking male. Only the pitch resembles the original speaker. Slight distortion and monotony are also present.
Female #1 (non-native)	High-Quality	0.58	0.69	Voice is very different from the original, with the prosody, tone, pitch, and accent, resembling a native English-speaking female. Slight distortion is present.

7.3. Findings

The highest-quality cloned voices are of Male #1 and Male #2 using Tortoise with High-Quality settings. The generated voices match these subjects' individual prosody and could be mistaken for genuine speech, aside from very minor distortion. Furthermore, some of the generated samples contain realistic speech artifacts, such as breathing or pauses, which increases believability and supports the potential of such samples to be utilized in an impersonation-based social engineering attack.

Despite its high similarity score, the generated samples for Male #3 are vastly dissimilar from the original voice. In addition, Female #1 scored very poorly for both similarity and human assessment. Therefore, in the configuration used in this study, the Tortoise system cannot be used to accurately generate voices with non-American accents. This will limit potential social engineers in terms of the individuals they can target with this framework, but adjusted attack strategies are still possible. One such strategy is to mimic a person close to the accented individual, such as their boss, family member, or bank representative.

Emotion proved to be a difficult vocal aspect to produce. Some of the generated sound files sounded monotonous and devoid of a normal rise-and-fall cadence. Although this characteristic could be useful in some cases, it is likely to arouse suspicion from a real target, especially if the monotony continues over a long passage of speech. However, when the reference sound files contained more emotion, our TTS models were able to create voices that possessed a level of emotion.

These results indicate that our proposed architecture is entirely feasible. That is, it is possible to develop a framework that enables a social engineer to perpetuate a highly convincing pretext attack solely by leveraging simple and highly accessible AI tools. Although our experiment leaves room for improvement regarding accents, emotion, and distortion, it demonstrates how easy it is to create a significant threat using basic AI resources. The goal

of our experiment was not to create a highly powerful system that can mimic any individual, rather it was to showcase that even a low-level AI framework can be used to carry out a believably deceptive pretext attack.

7.4. Impact on Society

The conversation-cloning capacity of our proposed architecture can be used to automate highly dangerous cyberattacks that rely on social engineering, including pig butchering. This framework both reduces the need for human labor at scam centers and provides an easily accessible way to mimic a target. Although open-source solutions tend to perform worse than closed models, as seen in the shortcomings of our study, this gap is closing quickly. AI tools like Gemma [32] and Coqui [88] are beginning to provide more robust accessible solutions. With access to stronger systems, the capabilities of threat actors can only increase.

As such tools advance, stories like those described in Section 5.2 may become more commonplace. As stated in Section 5.3, awareness may be our best defense against automated crime. Even so, this tactic can be a double-edged sword, as scam fears can diminish the fundamental trust that we place in those around us [6]. Individuals may begin questioning their own families and friends in desperate attempts to discern fact from fiction.

8. Discussion

Chatbots and GenAI have come a long way from the days of ELIZA. In 1966, "artificial intelligence" was merely an experiment to investigate how a human could interact with a machine [1]. Today, this basis has proven to be commercially viable for hundreds of millions of users. It may be a near reality that people will be able to communicate with chatbots in real-time, as if speaking to another human.

However, this study demonstrates that different areas of AI can still be abused despite the safeguards put in place. Contemporary powerful copilots possess strong abilities that are protected by extensive red teaming efforts. However, when combined with voice cloning, image generation, and other forms of generative technology, copilots can be a potent weapon in the hands of threat actors. Criminals who may not have been previously capable of committing a social engineering attack may become highly qualified through the use of such powerful AI tools. As innovation continues, tools such as the system developed in this paper will be better able to mimic their real counterparts. In the near future, generated content may reach a point where it is indistinguishable from reality.

As a result, robust safeguards against multi-part scam architectures must be established. Although cybersecurity awareness is a solid route for establishing threat recognition in the general public, preventive regulatory defense is a critical scheme that needs to be established. One such solution is comprehensive activity monitoring across all AI toolkits. By logging users' activity across different platforms, potential malicious behavior can be identified, investigated, and prevented from being deployed in a real-world attack. Although this detection is difficult, given the subtlety and convolution of modern attacks, AI-led systems can be trained to recognize patterns of impersonation and deceit in a path of usage logs across multiple generative resources. The feat of tracking user–model interaction across all platforms, including open-source tuning and closed-source prompting, is highly difficult, as it requires overarching moderation involving many dozens of companies and models. However, we believe it is a necessary integration in order to prevent highly orchestrated attacks that are undetectable when analyzing single-platform usage.

In terms of future work, additional research on offensive techniques is necessary to develop robust attack pattern detection. Additionally, red teaming efforts must begin their expansion across generative modalities and company domains to begin protecting against

the malicious combination of AI resources. With the right oversight and direction, it is possible to create digital assistants that benefit us all and are impossible to be used for harm.

Author Contributions: Conceptualization, H.P.D.V. and F.A.; methodology, H.P.D.V. and F.A.; software, H.P.D.V.; validation, H.P.D.V., F.A., J.W., and T.H.A.; formal analysis, H.P.D.V., F.A., and J.W.; investigation, H.P.D.V.; resources, H.P.D.V. and F.A.; data curation, H.P.D.V.; writing—original draft preparation, H.P.D.V.; writing—review and editing, J.W., F.A., and T.H.A.; visualization, J.W.; supervision, F.A.; project administration, F.A.; funding acquisition, F.A. All authors have read and agreed to the published version of the manuscript.

Funding: This research was supported by the U.S. National Science Foundation (Award#: 2319803). Opinions, findings, and conclusions are those of the authors and do not necessarily reflect the views of the NSF.

Institutional Review Board Statement: Ethical review and approval were waived for this study due to exemption under 45 C.F.R. § 46.104.

Informed Consent Statement: Written informed consent for participation in research was obtained from all subjects involved in the study. Written informed consent was also obtained from all subjects for the publication of this paper.

Data Availability Statement: The raw data supporting the conclusions of this article will be made available by the authors on request, to support the privacy of our study participants.

Conflicts of Interest: The authors declare no conflicts of interest. The funders had no role in the design of the study; in the collection, analyses, or interpretation of data; in the writing of the manuscript; or in the decision to publish the results.

Abbreviations

The following abbreviations are used in this manuscript:

LLM	Large Language Model
TTS	Text-To-Speech
AI	Artificial Intelligence
GenAI	Generative AI
NLP	Natural Language Processing
RNN	Recurrent Neural Network
LSTM	Long Short-Term Memory
LAMBADA	Language Modeling Broadened to Account for Discourse Aspects
VLM	Visual Language Model
RLHF	Reinforcement Learning from Human Feedback
GPT	Generative Pre-trained Transformer
CRM	Customer Relationship Management
NLU	Natural Language Understanding
AFF	Advance Fee Fraud
BiLSTM	Bidirectional Long Short-Term Memory
CNN	Convolutional Neural Network
NER	Named Entity Recognition
CRF	Confitional Random Field
SSO	Single Sign-On
NIST	National Institute of Science and Technology
SV2TTS	Transfer Learning from Speaker Verification to Multispeaker Text-To-Speech Synthesis
KNN	K-Nearest Neighbor
GCN	Graph Convolutional Network

References

1. Weizenbaum, J. *Computer Power and Human Reason*; W.H. Freeman and Company: New York, NY, USA, 1976.
2. Ziegler, A.; Kalliamvakou, E.; Li, X.A.; Rice, A.; Rifkin, D.; Simister, S.; Sittampalam, G.; Aftandilian, E. Productivity assessment of neural code completion. In Proceedings of the MAPS 2022: 6th ACM SIGPLAN International Symposium on Machine Programming, New York, NY, USA, 13 June 2022; pp. 21–29. [CrossRef]
3. Royzen, M. *Phind Model Beats GPT-4 at Coding, with GPT-3.5-like Speed and 16k Context*; Technical Report; Hello Cognition Inc.: San Francisco, CA, USA, 2023.
4. Zadeh, P.M.; Sattler, D. Improving Accessibility and Readability of Survey Reports in Digital Health Platforms using Conversational AI. In Proceedings of the 2023 IEEE International Conference on Bioinformatics and Biomedicine (BIBM), Istanbul, Turkiye, 5–8 December 2023; pp. 4987–4989. [CrossRef]
5. Microsoft. Microsoft Copilot: Satya Nadella at Microsoft Ignite 2023. 2023. Available online: https://www.youtube.com/watch?v=YEfnbXlcU_M&ab_channel=Microsoft (accessed on 15 February 2024).
6. CBS News. How Phone Scammers Are Using AI to Imitate Voices. 2023. Available online: https://www.youtube.com/watch?v=pJZYd_65xs4&ab_channel=CBSNews (accessed on 13 October 2024).
7. Shoaib, M.R.; Wang, Z.; Ahvanooey, M.T.; Zhao, J. Deepfakes, Misinformation, and Disinformation in the Era of Frontier AI, Generative AI, and Large AI Models. In Proceedings of the 2023 International Conference on Computer and Applications (ICCA), Cairo, Egypt, 28–30 November 2023; pp. 1–7. [CrossRef]
8. Zhou, J.; Zhang, Y.; Luo, Q.; Parker, A.G.; De Choudhury, M. Synthetic Lies: Understanding AI-Generated Misinformation and Evaluating Algorithmic and Human Solutions. In Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems, New York, NY, USA, 23–28 April 2023. [CrossRef]
9. Levantino, F.P. Generative and AI-powered oracles: "What will they say about you?". *Comput. Law Secur. Rev.* **2023**, *51*, 105898. [CrossRef]
10. Sugunaraj, N.; Ramchandra, A.R.; Ranganathan, P. Cyber Fraud Economics, Scam Types, and Potential Measures to Protect U.S. Seniors: A Short Review. In Proceedings of the 2022 IEEE International Conference on Electro Information Technology (EIT), Mankato, MN, USA, 19–21 May 2022; pp. 623–627. [CrossRef]
11. Ferrara, E. GenAI against humanity: Nefarious applications of generative artificial intelligence and large language models. *J. Comput. Soc. Sci.* **2024**, *7*, 549–569. [CrossRef]
12. Kang, D.; Li, X.; Stoica, I.; Guestrin, C.; Zaharia, M.; Hashimoto, T. Exploiting Programmatic Behavior of LLMs: Dual-Use Through Standard Security Attacks. *arXiv* **2023**, arXiv:2302.05733. [CrossRef]
13. OpenAI. GPT-4 Technical Report. *arXiv* **2023**, arXiv:2303.08774. [CrossRef]
14. Chen, H.; Magramo, K. Finance Worker Pays Out $25 Million After Video Call with Deepfake 'Chief Financial Officer'. 2024. Available online: https://www.cnn.com/2024/02/04/asia/deepfake-cfo-scam-hong-kong-intl-hnk/index.html (accessed on 8 June 2024).
15. OpenAI. March 20 ChatGPT Outage: Here's What Happened. 2023. Available online: https://openai.com/blog/march-20-chatgpt-outage (accessed on 8 June 2024).
16. Vassilev, A.; Oprea, A.; Fordyce, A.; Anderson, H. Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations. 2024. Available online: https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-2e2023.pdf (accessed on 6 April 2024).
17. Dendersky, A. What Is an AI Copilot. 2024. Available online: https://www.salesforce.com/blog/ai-copilot/ (accessed on 15 April 2025).
18. Weizenbaum, J. ELIZA—A Computer Program for the Study of Natural Language Communication between Man and Machine (1966). In *Ideas That Created the Future: Classic Papers of Computer Science*; MIT Press: Cambridge, MA, USA, 2021. [CrossRef]
19. Russell, S.J.; Norvig, P. *Artificial Intelligence: A Modern Approach*; Pearson: Noida, India, 2009.
20. Jozefowicz, R.; Vinyals, O.; Schuster, M.; Shazeer, N.; Wu, Y. Exploring the Limits of Language Modeling. *arXiv* **2016**, arXiv:1602.02410. [CrossRef]
21. Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, Ł.; Polosukhin, I. Attention Is All You Need. *arXiv* **2023**, arXiv:1706.03762.
22. Gemini Team. Gemini: A Family of Highly Capable Multimodal Models. *arXiv* **2023**, arXiv:2312.11805. [CrossRef]
23. Touvron, H.; Martin, L.; Stone, K.; Albert, P.; Almahairi, A.; Babaei, Y.; Scialom, T. Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv* **2023**, arXiv:2307.09288. [CrossRef]
24. Jacob, B.; Kligys, S.; Chen, B.; Zhu, M.; Tang, M.; Howard, A.; Adam, H.; Kalenichenko, D. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. *arXiv* **2017**, arXiv:1712.05877.
25. Xiao, G.; Lin, J.; Seznec, M.; Wu, H.; Demouth, J.; Han, S. *SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models*; PMLR: New York, NY, USA, 2023.
26. Zellers, R.; Holtzman, A.; Bisk, Y.; Farhadi, A.; Choi, Y. HellaSwag: Can a Machine Really Finish Your Sentence? *arXiv* **2019**, arXiv:1905.07830. [CrossRef]

27. Paperno, D.; Kruszewski, G.; Lazaridou, A.; Pham, N.Q.; Bernardi, R.; Pezzelle, S.; Baroni, M.; Boleda, G.; Fernández, R. The LAMBADA dataset: Word prediction requiring a broad discourse context. *arXiv* **2016**, arXiv:1606.06031. [CrossRef]
28. Chen, M.; Tworek, J.; Jun, H.; Yuan, Q.; de Oliveira Pinto, H.P.; Kaplan, J.; Edwards, H.; Burda, Y.; Joseph, N.; Brockman, G.; et al. Evaluating Large Language Models Trained on Code. *arXiv* **2021**, arXiv:2107.03374. [CrossRef]
29. Wang, J.; Liu, B. Recruitment Fraud Detection Method Based on Crowdsourcing and Multi-feature Fusion. In Proceedings of the 2022 5th International Conference on Artificial Intelligence and Big Data (ICAIBD), Chengdu, China, 27–30 May 2022; pp. 267–273. [CrossRef]
30. Hewett, J.; Leeke, M. Developing a GPT-3-Based Automated Victim for Advance Fee Fraud Disruption. In Proceedings of the 2022 IEEE 27th Pacific Rim International Symposium on Dependable Computing (PRDC), Beijing, China, 28 November–1 December 2022; pp. 205–211. [CrossRef]
31. Reynolds, L.; McDonell, K. Prompt Programming for Large Language Models: Beyond the Few-Shot Paradigm. *arXiv* **2021**, arXiv:2102.07350. [CrossRef]
32. Gemini Team. Gemma: Open Models Based on Gemini Research and Technology. *arXiv* **2024**, arXiv:2403.08295. [CrossRef]
33. Hugging Face. Hugging Face. Available online: https://huggingface.co/ (accessed on 18 September 2024).
34. LM Studio. LM Studio. 2023. Available online: https://lmstudio.ai/ (accessed on 10 August 2024).
35. Zhang, Y.; Sun, S.; Galley, M.; Chen, Y.C.; Brockett, C.; Gao, X.; Gao, J.; Liu, J.; Dolan, B. DialoGPT: Large-Scale Generative Pre-training for Conversational Response Generation. *arXiv* **2020**, arXiv:1911.00536.
36. Papineni, K.; Roukos, S.; Ward, T.; Zhu, W.J. BLEU: A method for automatic evaluation of machine translation. In Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL), Philadelphia, PA, USA, 6–12 July 2002; pp. 311–318.
37. Banerjee, S.; Lavie, A. METEOR: An Automatic Metric for MT Evaluation with Improved Correlation with Human Judgments. In Proceedings of the Workshop Intrinsic Extrinsic Measures Measures MT Summarization, 43rd Annual Meeting Association Computational Linguistics (ACL), Ann Arbor, MI, USA, 29 June 2005; pp. 65–72.
38. Doddington, G. Automatic evaluation of machine translation quality using n-gram co-occurrence statistics. In Proceedings of the 2nd International Conference on Human Language Technology Research (HLT), San Diego, CA, USA, 24–27 March 2002; pp. 138–145.
39. Alayrac, J.B.; Donahue, J.; Luc, P.; Miech, A.; Barr, I.; Hasson, Y.; Lenc, K.; Mensch, A.; Millican, K.; Reynolds, M.; et al. Flamingo: A Visual Language Model for Few-Shot Learning. *arXiv* **2022**, arXiv:2204.14198. [CrossRef]
40. Roziere, B.; Gehring, J.; Gloeckle, F.; Sootla, S.; Gat, I.; Tan, X.E.; Adi, Y.; Liu, J.; Remez, T.; Rapin, J. Code Llama: Open Foundation Models for Code. *arXiv* **2023**, arXiv:2308.12950.
41. Chen, X.; Wang, X.; Changpinyo, S.; Piergiovanni, A.; Padlewski, P.; Salz, D.; Goodman, S.; Grycner, A.; Mustafa, B.; Beyer, L.; et al. PaLI: A Jointly-Scaled Multilingual Language-Image Model. *arXiv* **2023**, arXiv:2209.06794.
42. Zhai, X.; Wang, X.; Mustafa, B.; Steiner, A.; Keysers, D.; Kolesnikov, A.; Beyer, L. LiT: Zero-Shot Transfer with Locked-Image Text Tuning. *arXiv* **2022**, arXiv:2111.07991.
43. Shazeer, N. Fast Transformer Decoding: One Write-Head is All You Need. *arXiv* **2019**, arXiv:1911.02150. [CrossRef]
44. Shazeer, N. GLU Variants Improve Transformer. *arXiv* **2020**, arXiv:2002.05202. [CrossRef]
45. Zhang, B.; Sennrich, R. Root Mean Square Layer Normalization. *arXiv* **2019**, arXiv:1910.07467. [CrossRef]
46. De Angelis, L.; Baglivo, F.; Arzilli, G.; Privitera, G.P.; Ferragina, P.; Tozzi, A.E.; Rizzo, C. ChatGPT and the rise of large language models: The new AI-driven infodemic threat in public health. *Front. Public Health* **2023**, *11*, 1166120. [CrossRef] [PubMed]
47. Jones, C.R.; Bergen, B.K. People cannot distinguish GPT-4 from a human in a Turing test. *arXiv* **2024**, arXiv:2405.08007.
48. OpenAI. *Video Generation Models as World Simulators*; Technical Report; OpenAI Corp: San Francisco, CA, USA, 2023.
49. Ramesh, A.; Pavlov, M.; Goh, G.; Gray, S.; Voss, C.; Radford, A.; Chen, M.; Sutskever, I. Zero-Shot Text-to-Image Generation. *arXiv* **2021**, arXiv:2102.12092.
50. Adiwardana, D.; Luong, M.T.; So, D.R.; Hall, J.; Fiedel, N.; Thoppilan, R.; Yang, Z.; Kulshreshtha, A.; Nemade, G.; Lu, Y.; et al. Towards a Human-like Open-Domain Chatbot. *arXiv* **2020**, arXiv:2001.09977.
51. Cleverbot. Cleverbot. 2024. Available online: https://www.cleverbot.com/ (accessed on 10 April 2024).
52. ICONIQ. Kuki.ai. 2024. Available online: https://www.kuki.ai/research (accessed on 10 April 2024).
53. Zhou, L.; Gao, J.; Li, D.; Shum, H.Y. The Design and Implementation of XiaoIce, an Empathetic Social Chatbot. *arXiv* **2019**, arXiv:1812.08989. [CrossRef]
54. So, D.R.; Liang, C.; Le, Q.V. The Evolved Transformer. *arXiv* **2019**, arXiv:1901.11117. [CrossRef]
55. Zhang, Y.; Han, W.; Qin, J.; Wang, Y.; Bapna, A.; Chen, Z.; Chen, N.; Li, B.; Axelrod, V.; Wang, G.; et al. Google USM: Scaling Automatic Speech Recognition Beyond 100 Languages. *arXiv* **2023**, arXiv:2303.01037. [CrossRef]
56. Salesforce. Salesforce Announces Einstein GPT, the World's First Generative AI for CRM. 2023. Available online: https://www.salesforce.com/news/press-releases/2023/03/07/einstein-generative-ai/ (accessed on 15 April 2025).

57. Microsoft. *Frequently Asked Questions: AI, Microsoft Copilot, and Microsoft Designer*; Technical Report; Microsoft Corporation: Washington, DC, USA, 2024.
58. Kesarwani, S.; Titiksha; Juneja, S. Student Chatbot System: A Review on Educational Chatbot. In Proceedings of the 2023 7th International Conference on Trends in Electronics and Informatics (ICOEI), Tirunelveli, India, 11–13 April 2023; pp. 1578–1583. [CrossRef]
59. Meshram, S.; Naik, N.; VR, M.; More, T.; Kharche, S. College Enquiry Chatbot using Rasa Framework. In Proceedings of the 2021 Asian Conference on Innovation in Technology (ASIANCON), Pune, India, 27–29 August 2021; pp. 1–8. [CrossRef]
60. Dakhel, A.M.; Majdinasab, V.; Nikanjam, A.; Khomh, F.; Desmarais, M.C.; Jiang, Z.M.J. GitHub Copilot AI pair programmer: Asset or Liability? *J. Syst. Softw.* **2023**, *203*, 111734. [CrossRef]
61. Dhyani, P.; Nautiyal, S.; Negi, A.; Dhyani, S.; Chaudhary, P. Automated API Docs Generator using Generative AI. In Proceedings of the 2024 IEEE International Students' Conference on Electrical, Electronics and Computer Science (SCEECS), Bhopal, India, 24–25 February 2024; pp. 1–6. [CrossRef]
62. Qiao, Z.; Zhang, C.; Du, G. Improving Cybersecurity Named Entity Recognition with Large Language Models. In Proceedings of the 2023 6th International Conference on Software Engineering and Computer Science (CSECS), Chengdu, China, 22–24 December 2023; pp. 1–6. [CrossRef]
63. Bhanushali, M.; Parekh, H.; Mistry, R.; Mane, Y. TAKA Cybersecurity Chatbot. In Proceedings of the 2023 International Conference on Advanced Computing Technologies and Applications (ICACTA), Mumbai, India, 6–7 October 2023; pp. 1–6. [CrossRef]
64. Heiding, F.; Schneier, B.; Vishwanath, A.; Bernstein, J.; Park, P.S. Devising and Detecting Phishing: Large Language Models vs. Smaller Human Models. *arXiv* **2023**, arXiv:2308.12287. [CrossRef]
65. Greshake, K.; Abdelnabi, S.; Mishra, S.; Endres, C.; Holz, T.; Fritz, M. Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. *arXiv* **2023**, arXiv:2302.12173.
66. Kromphardt, T.; Po, G. Broken Dreams and Piggy Banks: Pig Butchering Crypto Fraud Growing Online. 2023. Available online: https://www.proofpoint.com/us/blog/threat-insight/broken-dreams-and-piggy-banks-pig-butchering-crypto-fraud-growing-online (accessed on 8 June 2024).
67. McNeil, A. Trash Talk: Pig Butchering and Conversational Attacks Were the Fastest Growing Mobile Threats of 2022. 2023. Available online: https://www.proofpoint.com/us/blog/pig-butchering-conversational-attacks-fastest-growing-mobile-threats-of-2022 (accessed on 8 June 2024).
68. Sumsub. Sumsub Research: Global Deepfake Incidents Surge Tenfold from 2022 to 2023. 2023. Available online: https://sumsub.com/newsroom/sumsub-research-global-deepfake-incidents-surge-tenfold-from-2022-to-2023/ (accessed on 8 June 2024).
69. Amezaga, N.; Hajek, J. Availability of Voice Deepfake Technology and its Impact for Good and Evil. In Proceedings of the 23rd Annual Conference on Information Technology Education (SIGITE '22), New York, NY, USA, 21–24 September 2022; pp. 23–28. [CrossRef]
70. Jia, Y.; Zhang, Y.; Weiss, R.J.; Wang, Q.; Shen, J.; Ren, F.; Chen, Z.; Nguyen, P.; Pang, R.; Moreno, I.L.; et al. Transfer Learning from Speaker Verification to Multispeaker Text-To-Speech Synthesis. *arXiv* **2019**, arXiv:1806.04558.
71. Oh, S.; Shon, T. Cybersecurity Issues in Generative AI. In Proceedings of the 2023 International Conference on Platform Technology and Service (PlatCon), Busan, Replic of Korea, 16–18 August 2023; pp. 97–100. [CrossRef]
72. Roy, S.S.; Thota, P.; Naragam, K.V.; Nilizadeh, S. From Chatbots to PhishBots?—Preventing Phishing scams created using ChatGPT, Google Bard and Claude. *arXiv* **2024**, arXiv:2310.19181.
73. Midjourney. Midjourney. Available online: https://www.midjourney.com/home (accessed on 28 June 2024).
74. Federal Bureau of Investigation. FBI Internet Crime Report 2022. 2022. Available online: https://www.ic3.gov/Media/PDF/AnnualReport/2022_IC3Report.pdf (accessed on 8 June 2024).
75. LastWeekTonight. Pig Butchering Scams: Last Week Tonight with John Oliver (HBO). 2024. Available online: https://www.youtube.com/watch?v=pLPpl2ISKTg&ab_channel=LastWeekTonight (accessed on 8 June 2024).
76. Sharma, M.; Singh, K.; Aggarwal, P.; Dutt, V. How well does GPT phish people? An investigation involving cognitive biases and feedback. In Proceedings of the 2023 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW), Delft, The Netherlands, 3–7 July 2023; pp. 451–457. [CrossRef]
77. City of New York. Tips on Artificial Intelligence Scams. 2024. Available online: https://www.nyc.gov/site/dca/consumers/artificial-intelligence-scam-tips.page (accessed on 24 March 2024).
78. Truong, T.C.; Diep, Q.B.; Zelinka, I. Artificial Intelligence in the Cyber Domain: Offense and Defense. *Symmetry* **2020**, *12*, 410. [CrossRef]
79. Krishnan, L.P.; Vakilinia, I.; Reddivari, S.; Ahuja, S. Scams and Solutions in Cryptocurrencies—A Survey Analyzing Existing Machine Learning Models. *Information* **2023**, *14*, 171. [CrossRef]

80. Meta. Expanding Our Open Source Large Language Models Responsibly. 2024. Available online: https://ai.meta.com/blog/meta-llama-3-1-ai-responsibility/ (accessed on 15 June 2025).
81. OpenAI. OpenAI Red Teaming Network. 2023. Available online: https://openai.com/index/red-teaming-network/ (accessed on 15 June 2025).
82. Google. Google's AI Red Team: The Ethical Hackers Making AI Safer. 2023. Available online: https://blog.google/technology/safety-security/googles-ai-red-team-the-ethical-hackers-making-ai-safer/ (accessed on 15 June 2025).
83. Betker, J. Tortoise-TTS. 2023. Available online: https://github.com/neonbjb/tortoise-tts (accessed on 26 August 2025).
84. Jemine, C. Real-Time Voice Cloning. Master's Thesis, University of Liège, Liège, Belgium, 2019.
85. Thorsten-Voice. XTTS FAQ | Interview with Josh Meyer from Coqui AI. 2023. Available online: https://www.youtube.com/watch?v=XsOM1WZ0k84_channel=Thorsten-Voice (accessed on 26 August 2025).
86. Resemble-AI. Resemblyzer. 2020. Available online: https://github.com/resemble-ai/Resemblyzer (accessed on 26 August 2025).
87. Betker, J. Better speech synthesis through scaling. *arXiv* **2023**, arXiv:2305.07243. [CrossRef]
88. coqui.ai. TTS. 2023. Available online: https://github.com/coqui-ai/TTS (accessed on 26 August 2025).

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.

Reproduced with permission of copyright owner. Further reproduction prohibited without permission.